Linux ACL Performance Analysis

Jamal Hadi Salim & Nabil Bitar

Outline

- Work Motivation
- Findings summary
  - Datapath performance (forwarding and host)
  - Two ACL workloads, Forwarding vs Host on IPv4 5-tuple Rulesets, comparing following technologies:
    - Iptables
    - Iptables + Ipset (with and without conntracking)
    - TC/flower
    - TC/ebpf
    - XDP/ebpf
- Experiments' Details and Results
- Next steps - discussion

Motivation

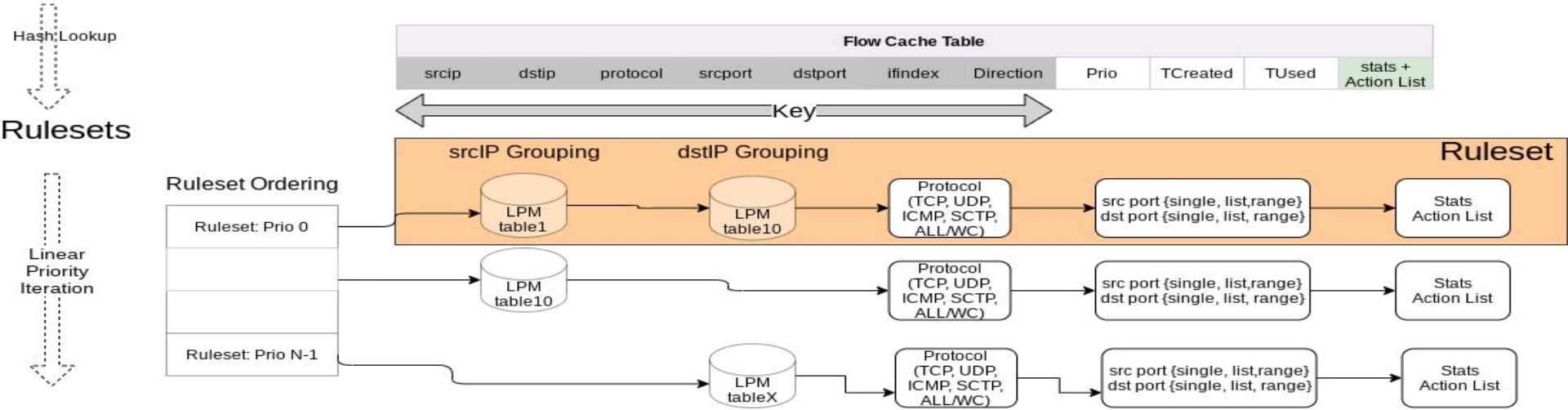
- To take a holistic approach to understand Linux ACL performance - A lot of good but incomplete and sometimes non-comparable information out there
  - Started with no assumptions of which subsystem was better
  - Had the generalized requirement on solutions to look like IPtables with IPSet and conntack

- Performance has direct $ cost impact. Lower performance, measured in throughput (and latency) per CPU cycle, means:
  - More CPUs/machines needed
  - More power consumed

ACL General Software Abstraction

Lookup Order:

1) *Optional* Flow Cache Table (exact match) (equivalent to Iptables *conntrack*)
2) If miss on FCT then search Ruleset Tables (equivalent to *Iptables* rules IP address grouping based on *IPSet*)
    a) On match update FCT
    b) On miss: Default rule



Experimental Setup

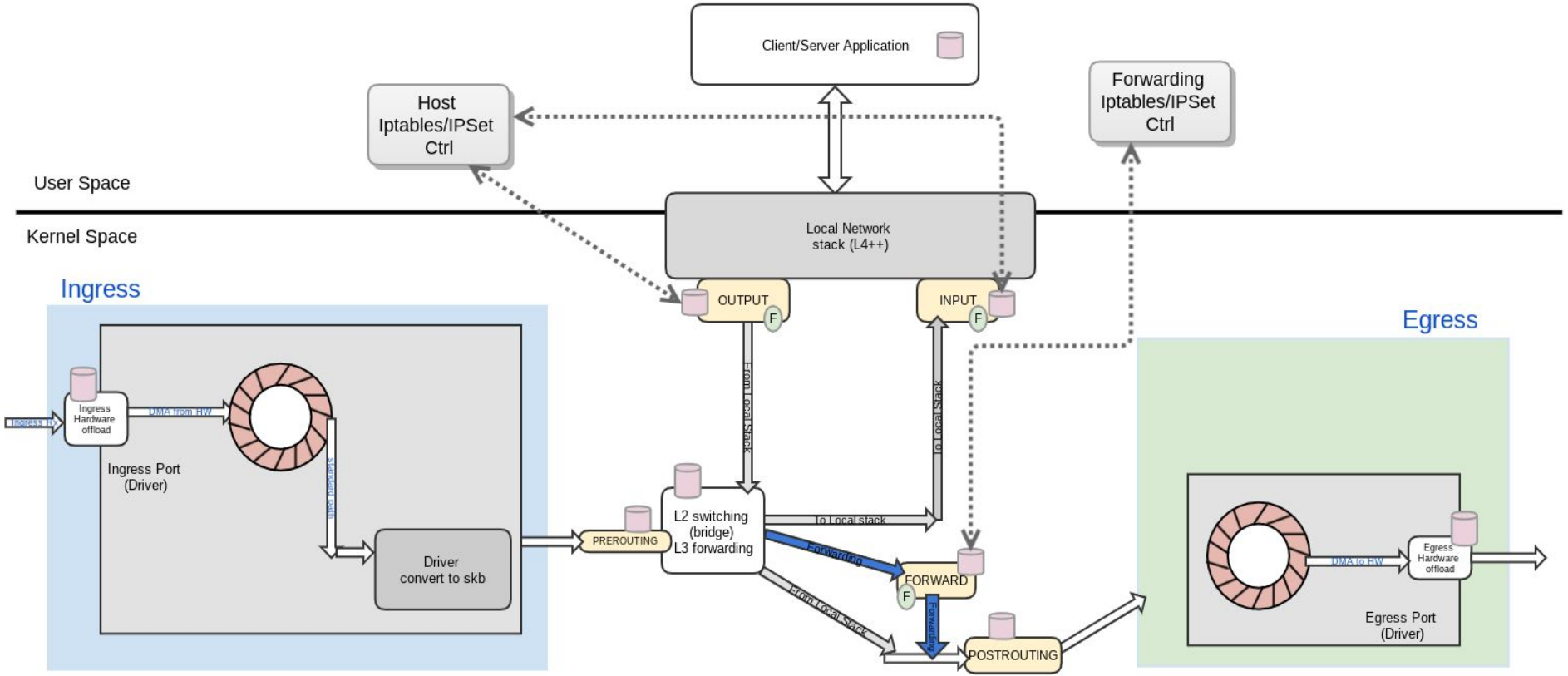
Reducing Experimental Variables

- Disable Turbo Boost
- CPU P+C-state Disabled
- Enhanced Intel® Speedstep® Tech Disabled
- Disable Hyper Threading
- Set scaling_governor to ‘performance’
- Turn off IRQbalance
- Disable Meltdown mitigation
- Turn off Network manager
- Turn-on isolcpus in the case of host based testing

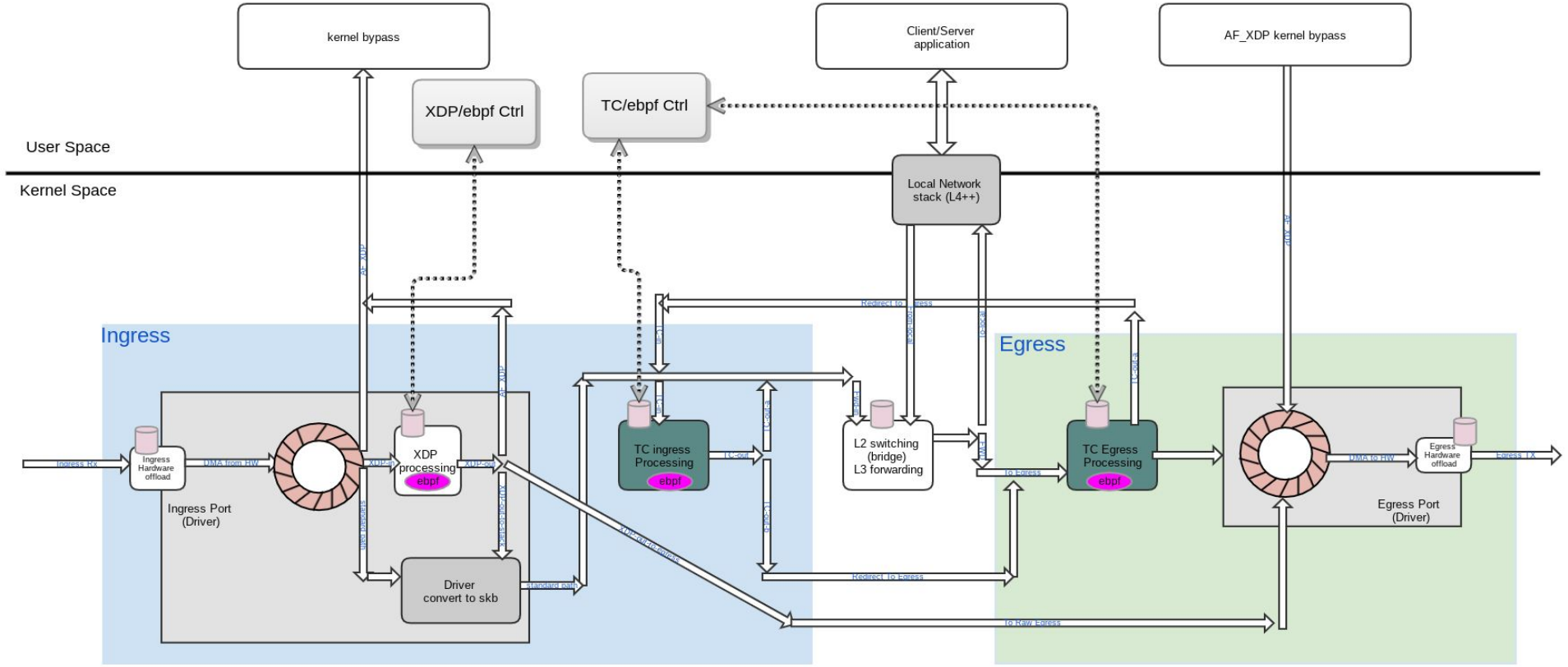
Workloads Tested

- Forwarding
- Host (ingress and Egress)
- For both workloads, tested for the worst case scenarios
  - In linear iteration equates to all rulesets are iterated and match on last entry
- Test Setup constituted
  - Baseline - no ACLs
  - For each ACL type,
    - vary ACL Ruleset count: 1, 7, 70, 700, 1400, 3000 Rulesets
    - Each ruleset with 50x50 CIDRs
    - Example: 3000 rulesets is 50x50x3000 individual rules
- We run 3 sets of each test
  - Perf was collected on one of them and we averaged the other two
  - Kernel: 5.13-rc6

Reminder: Iptables/IPset



Reminder: TC/XDP/Ebpf



30 Minutes Of Brevity...

- We are not going to discuss control path results (see backup)
  - Summary (throughput in delete/add operations): ebpf >>> IPSet >> IPTables
- We are not going to show IPtables without IPSet
  - After the 70 ruleset threshold performance deteriorated so much we dropped testing
  - The Linear iteration of IPtables with IPSet is a good representation of IPtables still
- We are not going to show data on Latency for Forwarding
  - TREX v2.82 produced inconsistent latency results
- We are reducing the amount of data we present on Flower
  - There is no easy fit into the ACL model that matches the Trio (IPtables/Set/conntrack)
    - We can make it work - but would require a more complex control plane
      - Overlapping rules and masks
  - Flower works well if you Fixed the mask per table
    - Deteriorates badly once you have different masks within same table

Tests Result Analysis

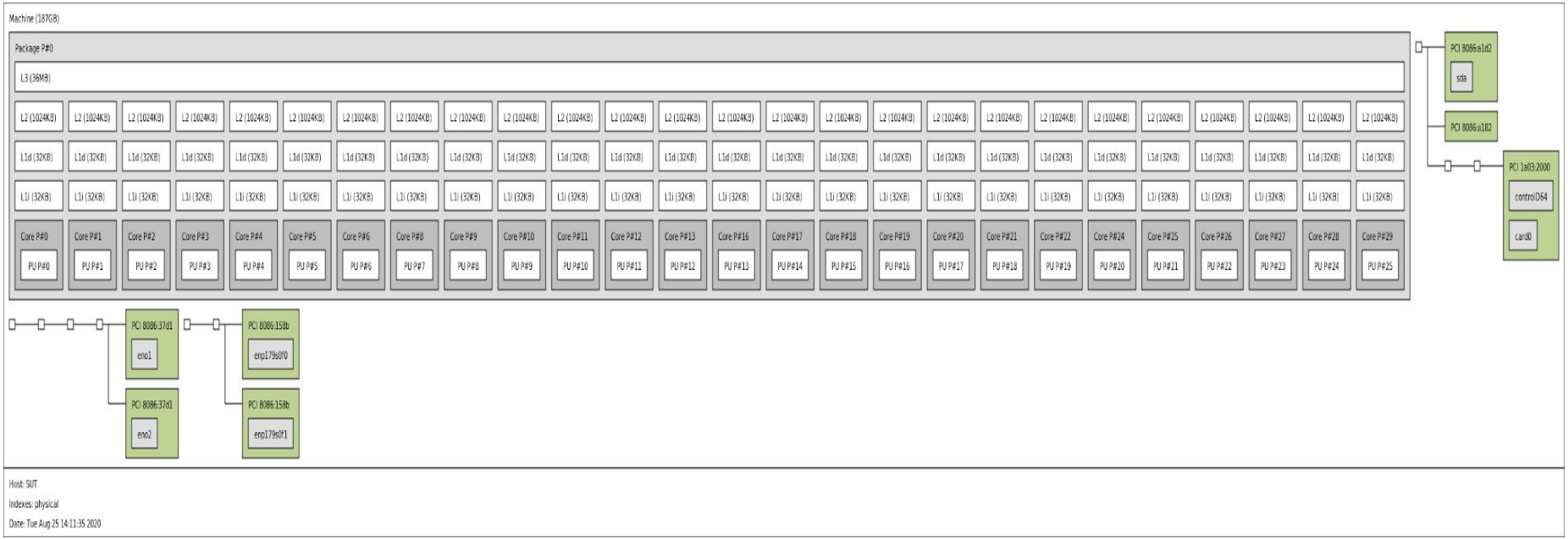
- We Collected a lot of data
- Looking at just one piece of data (example PPS or CPS) does not tell the whole story
- We came up with two metrics
  - **PPS/CPU**: normalizes the achieved packets/sec over the CPU utilization to achieve that rate
  - **PPS/Latency**: normalizes the achieved packets/sec over the observed Latency

Some Bias...

- We let packets go to the stack
  - XDP/TC can bypass the stack
- We are only considering the caching effect for eBPF
  - Conntracking does full bidirectional state tracking
    - Also takes 2-3 packets (vs 1 for ebpf) to activate

Datapath Testing

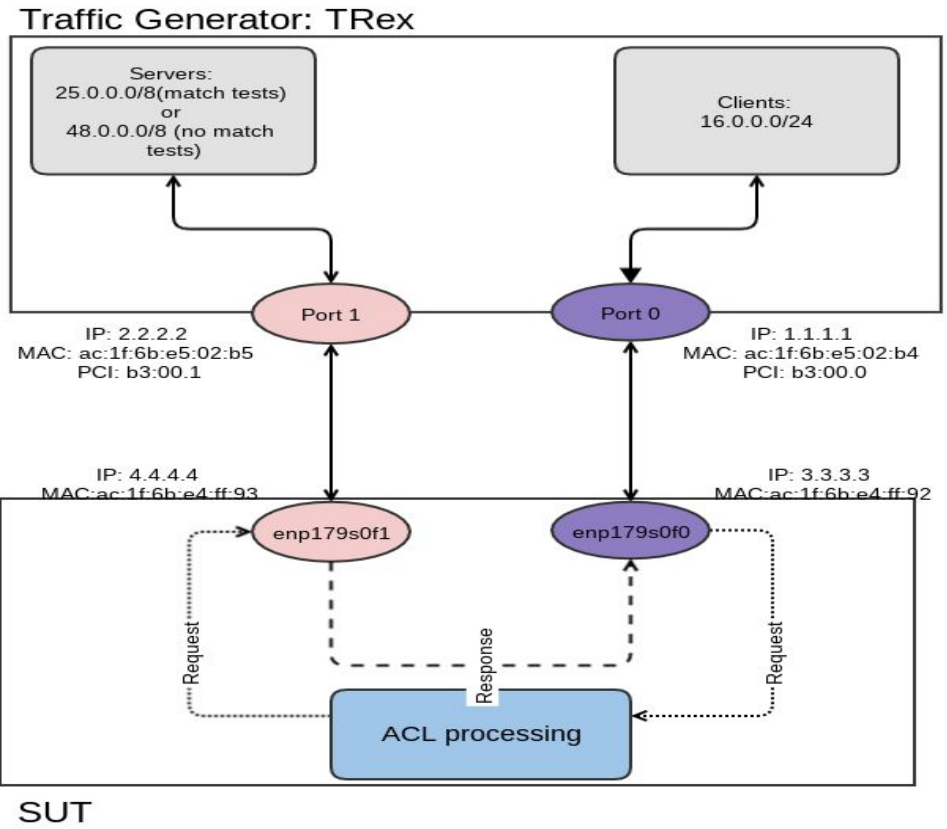
Test Environment: Hardware



- Intel(R) Xeon(R) Gold 6230R CPU @ 2.10GHz
- 196G RAM
- Turn off Hyperthreading (shows 26 CPUs)
- Only one Numa node

- NIC XXV710 dual-port NIC is connected on the PCI-E 3.0 x16 bus connected directly on the processor core
  - Dual 25Gbps

Forwarding Setup: Traffic Generator



TREX v 2.82
Need to test statefulness (conntrack) cost:

- Traffic generator running stateful TCP traffic
  - Real BSD TCP over DPDK

Traffic profile

- Symmetric Transactional bidirectional data
  - Neither elephant nor mouse
    - 300 packets total at ~80B
  - Simulating clients and servers on both sides justifies symmetric
  - Send at 60K connections per sec

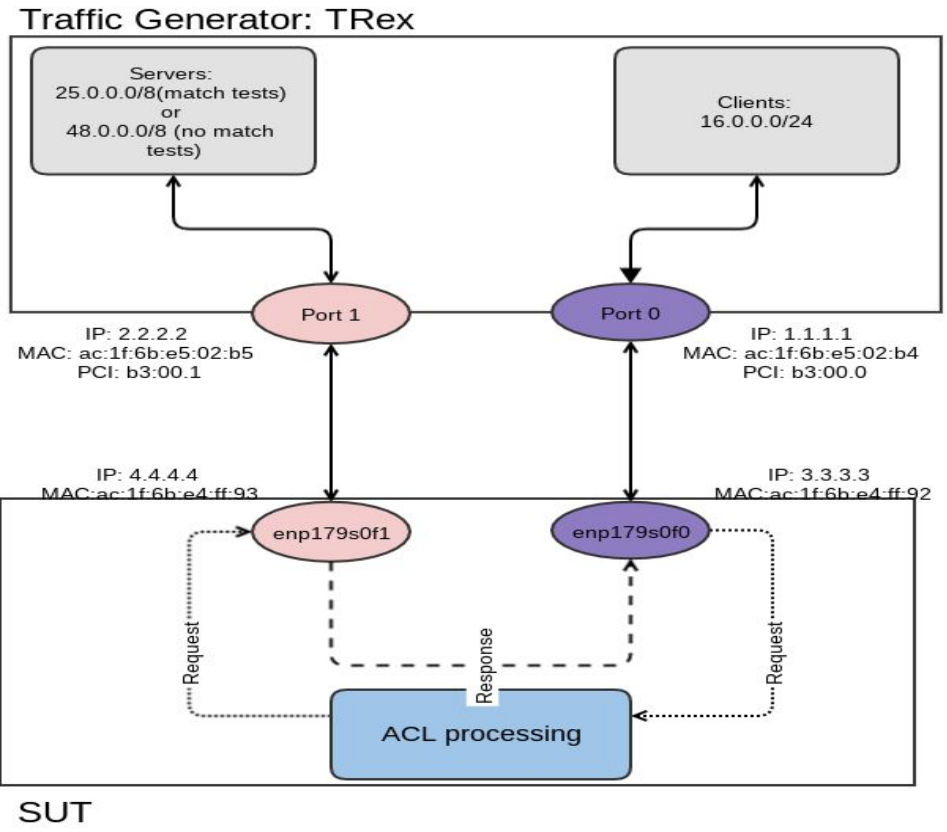
For both Clients and servers (TX and RX)

- Measure pps, cps, bps, latency, drops

Trex Latency measurement is out of band

- ICMP with embedded timestamps

Forwarding Setup: System Under Test



Kernel: 5.13-rc6 (with and without NF)

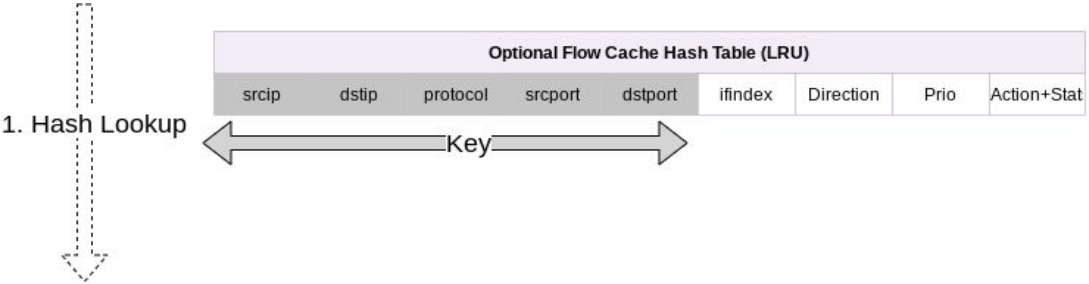
Symmetric RSS Load balancing

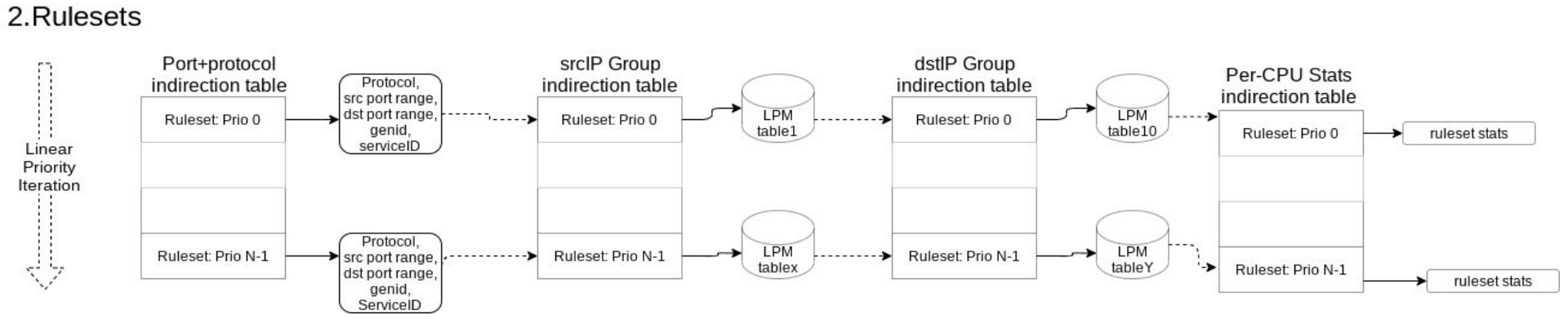
- Alongside generated traffic pattern allows nice balancing across CPUs

Test Setup

- Baseline
- For each ACL type
  - vary ACL Ruleset count
    - 1, 7, 70, 700, 1400, 3000
    - Each ruleset with 50x50 CIDRs
- Collect CPU and perf profiles

Ptables Abstraction



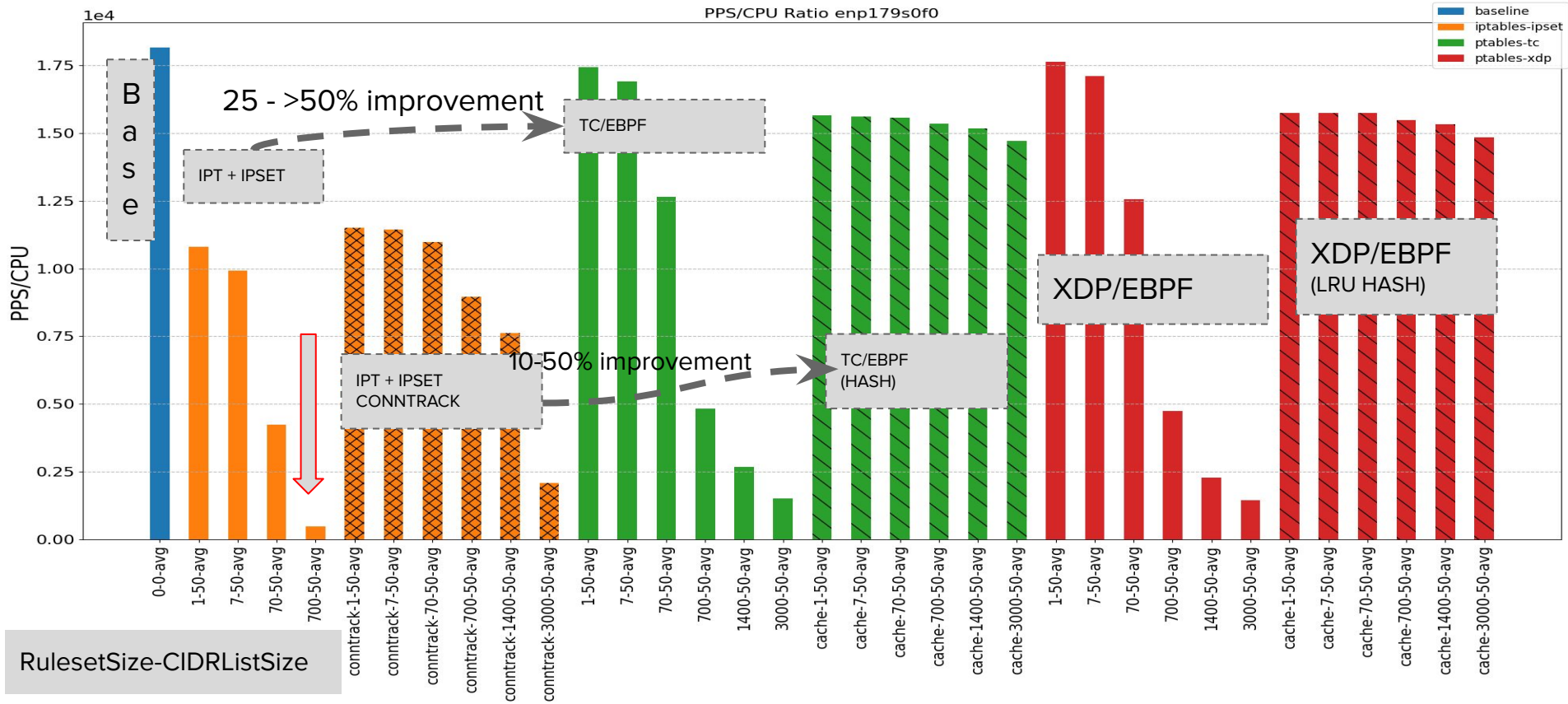


Forwarding Test Profile: Connection Tracking Setup

- Set Cache to be 8M
  - We generate about 6M flows total
    - Avoids having to deal with cache eviction

Forwarding: NF compiled into Kernel

Forwarding: NF ON PPS/CPU (Higher Better)



Forwarding Results: NF ON

- Ruleset Linear iteration as expected got worse with rules
  - TC/XDP ebpf did better than IPtables/IPSet
  - IPtables/IPSet results after 700 ruleset intolerable
  - No difference between XDP and TC because packets go to stack
- Caching improved the performance of linear search
  - After 70 rulesets IPtables conntracking was >=3x than without
  - Conntracking still deteriorated with increasing rulesets
    - Likely the effect of the expensive linear iptables rules scan
  - Ebpf both at TC and XDP with caching outperformed conntracking
    - After 700 rulesets by 2x

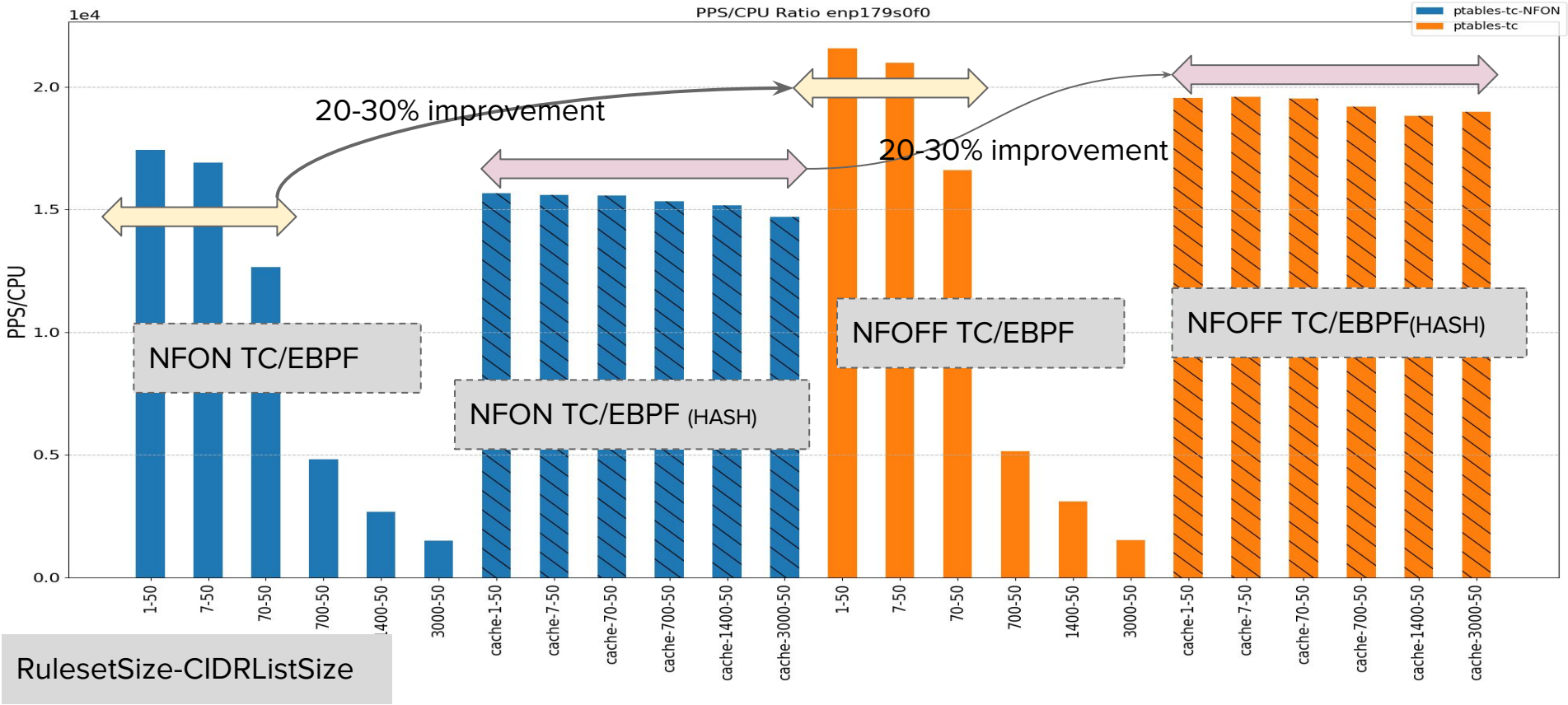
Forwarding NF compiled out of Kernel

Challenges With NF Compiled In

NFON: TC/Ebpf 70 Rulesets Perf Output

```
22.61%  ksoftirqd/1  bpf_prog_5baaa2b0c31f333f  [k] bpf_prog_5baaa2b0c31f333f
13.43%  ksoftirqd/1  [kernel.kallsyms]         [k] ipt_do_table
10.61%  ksoftirqd/1  [kernel.kallsyms]         [k] _raw_spin_lock
 3.79%  ksoftirqd/1  [i40e]                    [k] i40e_lan_xmit_frame
 3.48%  ksoftirqd/1  [kernel.kallsyms]         [k] fib_table_lookup
 3.23%  ksoftirqd/1  [i40e]                    [k] i40e_napi_poll
 1.88%  ksoftirqd/1  [kernel.kallsyms]         [k] fib_lookup_good_nhc
 1.87%  ksoftirqd/1  [kernel.kallsyms]         [k] __netif_receive_skb_core
 1.57%  ksoftirqd/1  [kernel.kallsyms]         [k] nf_hook_slow
 1.53%  ksoftirqd/1  [kernel.kallsyms]         [k] ip_route_input_slow
 1.49%  ksoftirqd/1  [kernel.kallsyms]         [k] __dev_queue_xmit
 1.29%  ksoftirqd/1  [kernel.kallsyms]         [k] ip_forward
 1.17%  ksoftirqd/1  [kernel.kallsyms]         [k] __fib_validate_source
 1.16%  ksoftirqd/1  [kernel.kallsyms]         [k] kmem_cache_free_bulk
 1.12%  ksoftirqd/1  [kernel.kallsyms]         [k] eth_type_trans
 1.10%  ksoftirqd/1  [kernel.kallsyms]         [k] ip_finish_output2
 1.08%  ksoftirqd/1  [kernel.kallsyms]         [k] kmem_cache_alloc
 1.08%  ksoftirqd/1  [kernel.kallsyms]         [k] __build_skb
```

Forwarding TC NF OFF vs NF ON PPS/CPU



Forwarding: Kernel NF Compiled out

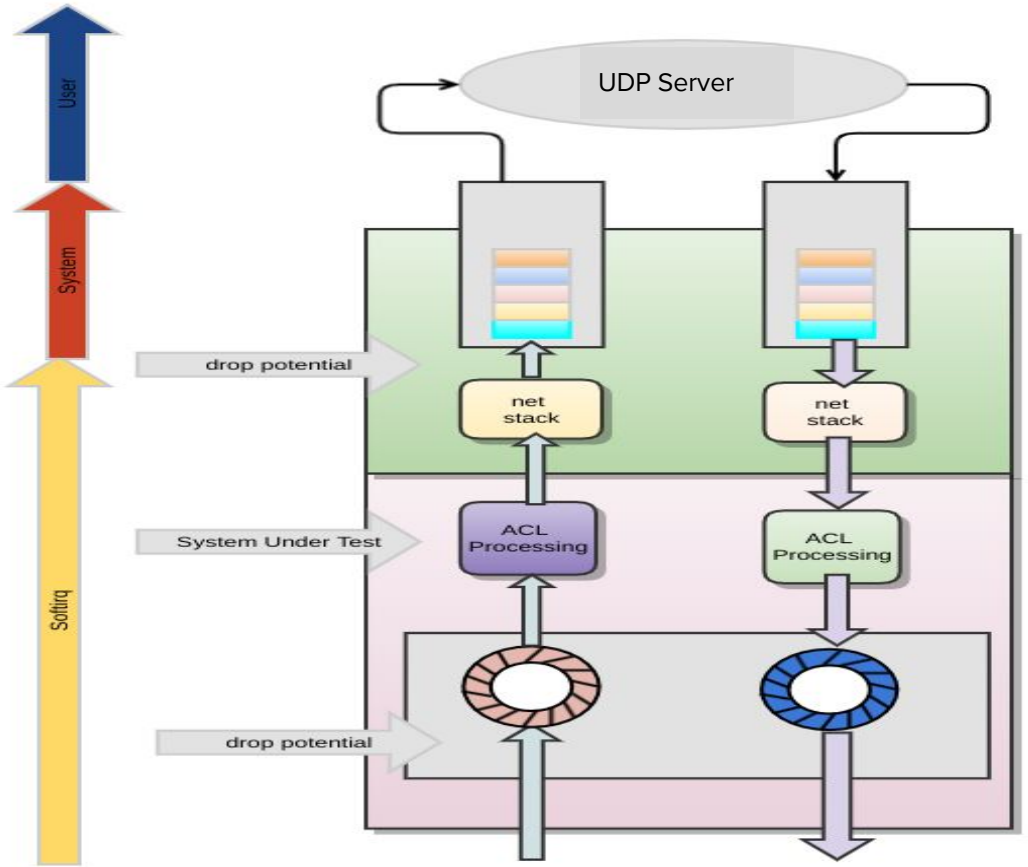
- Avoiding the NF tax
  - NF hooks are always hit when forwarding
  - Only tc/flower, tc/ebpf and XDP/ebpf can run in this kernel
- ~20-30% performance improvement vs. NF compiled in for TC and XDP
  - In addition to improvement over IPset

Host Workload

Host Traffic Test Setup

- Single threaded UDP client-server setup with ability to measure end to end latency
  - Client and server hardware same as forwarding tests
- PTP activated to get higher granularity in latency measurement
- Sent 1K packets upto about the MLFRR
- Collected PPS, BPS, latency on traffic generator
- Collected CPU and perf on SUT

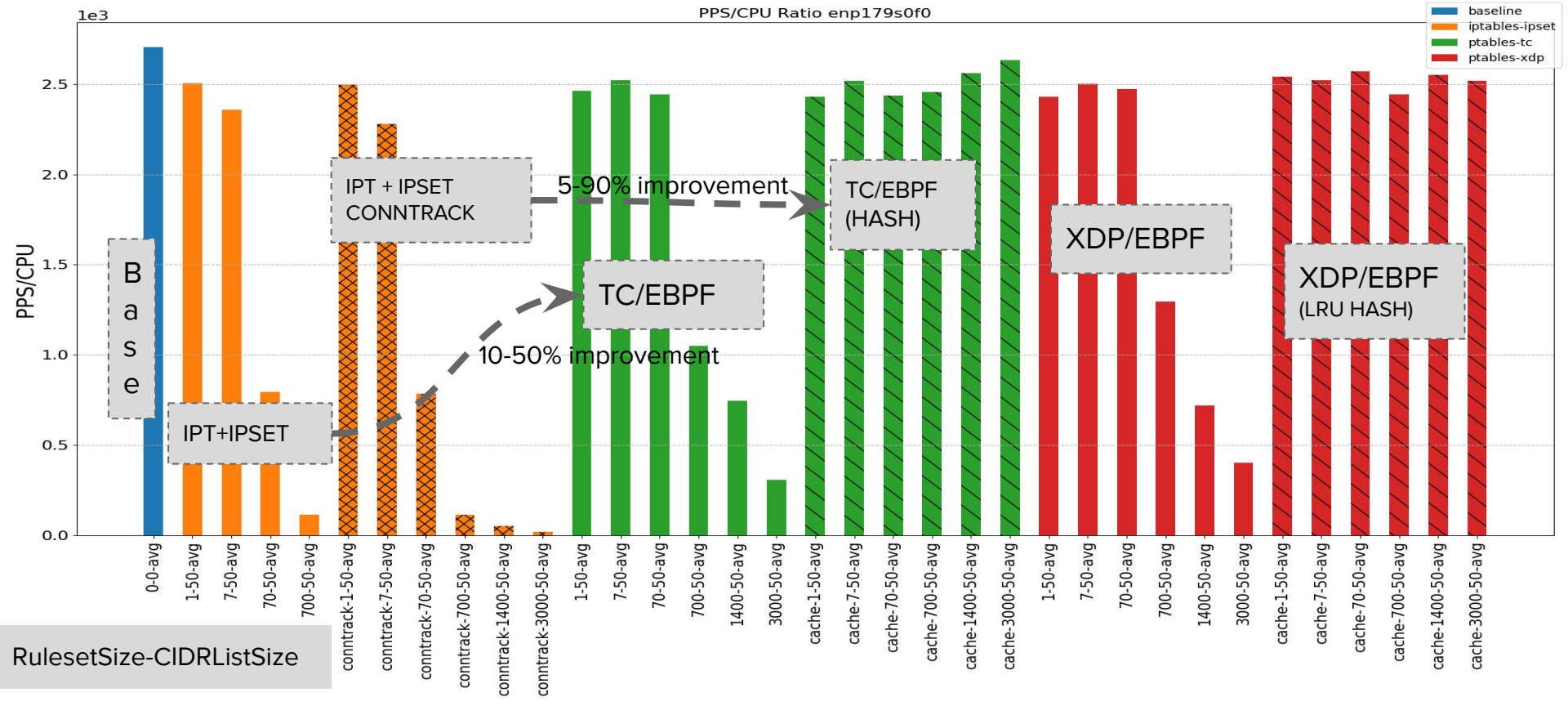
Host: SUT Ingress Processing



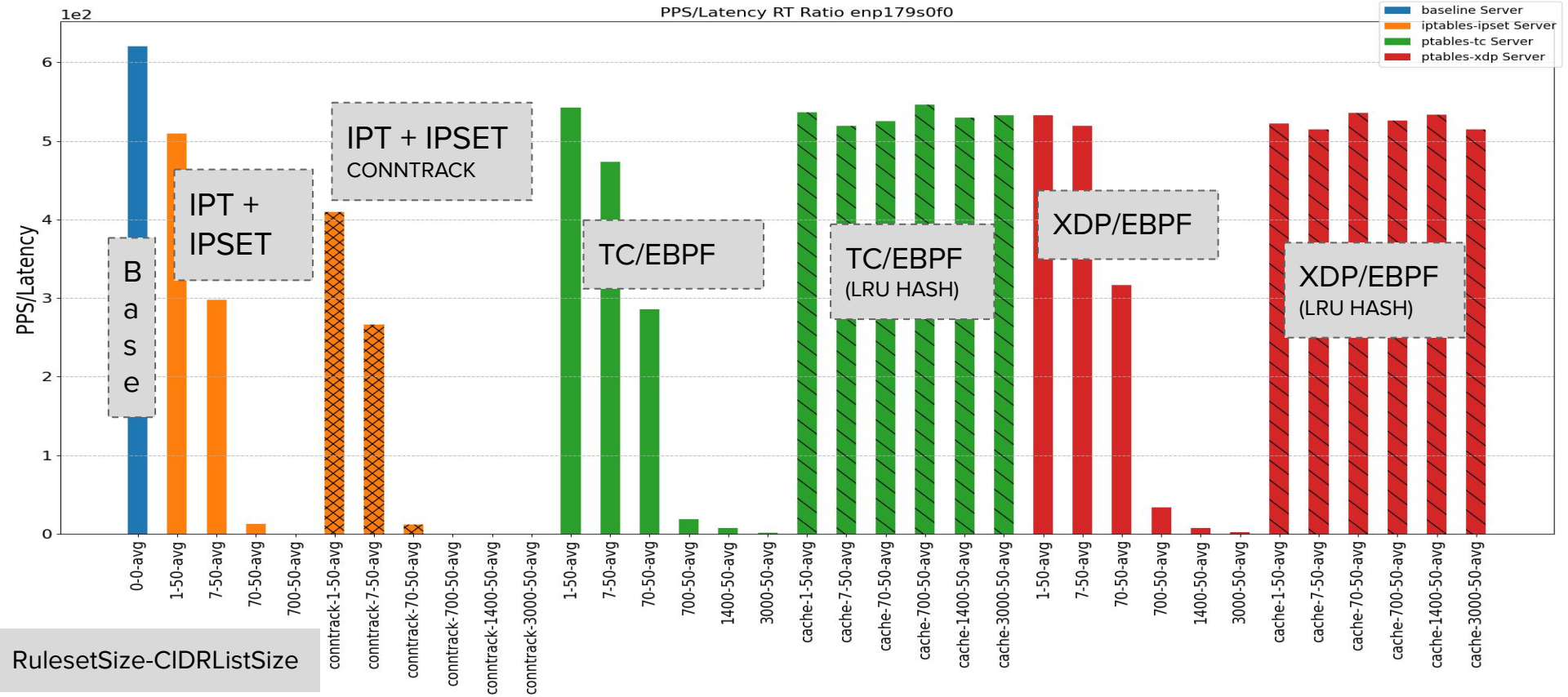
- NIC irq tied to one core
- Single threaded UDP app tied to second core
- Sending rate set to MLFRR
  - Drops [0, 0.0001%]
- Capture:
  - Throughput
  - latency
  - CPU breakdown
  - Drops

Host Ingress NF compiled Into Kernel

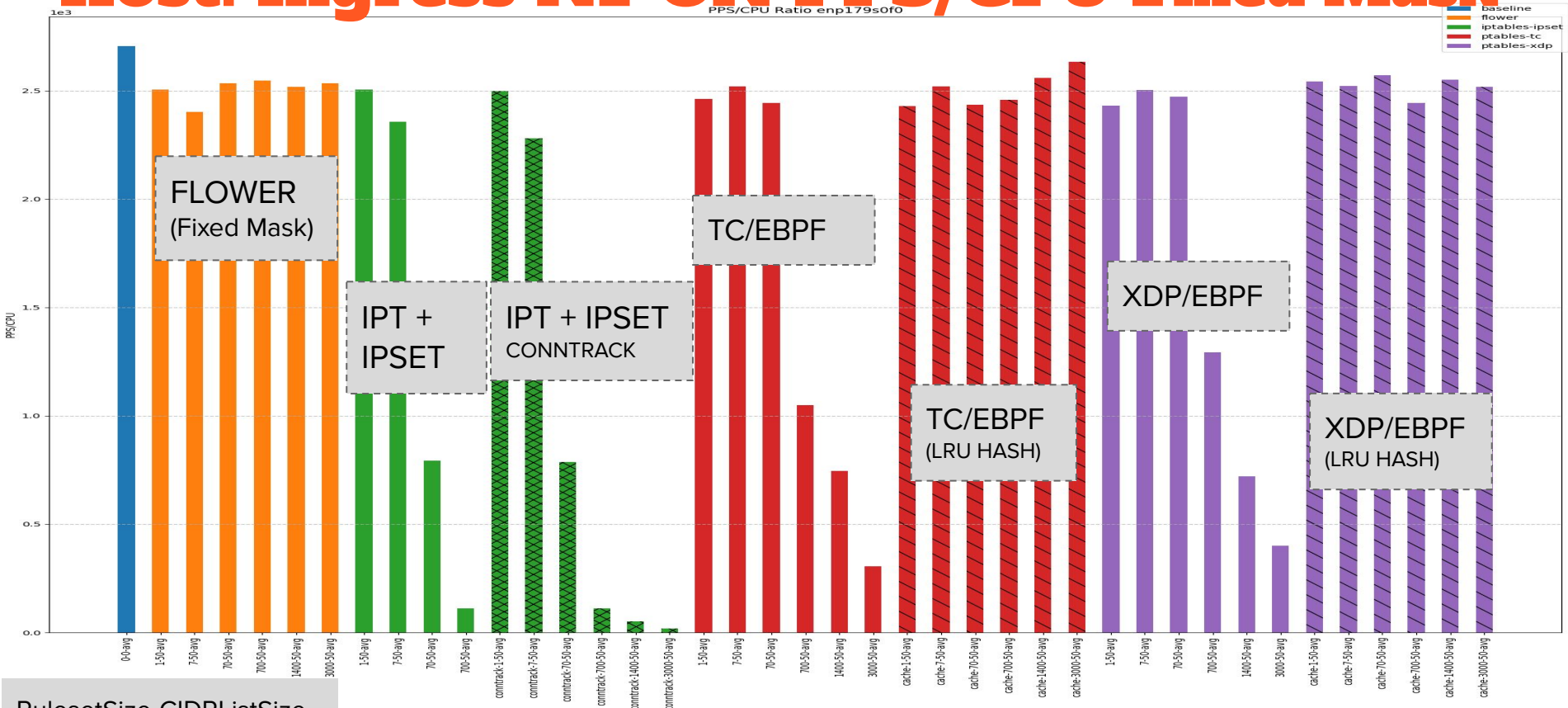
Host: Ingress NF ON PPS/CPU (Higher Better)



Host: Ingress NF ON PPS/Latency (Higher Better)



Host: Ingress NF ON PPS/CPU Fixed Mask

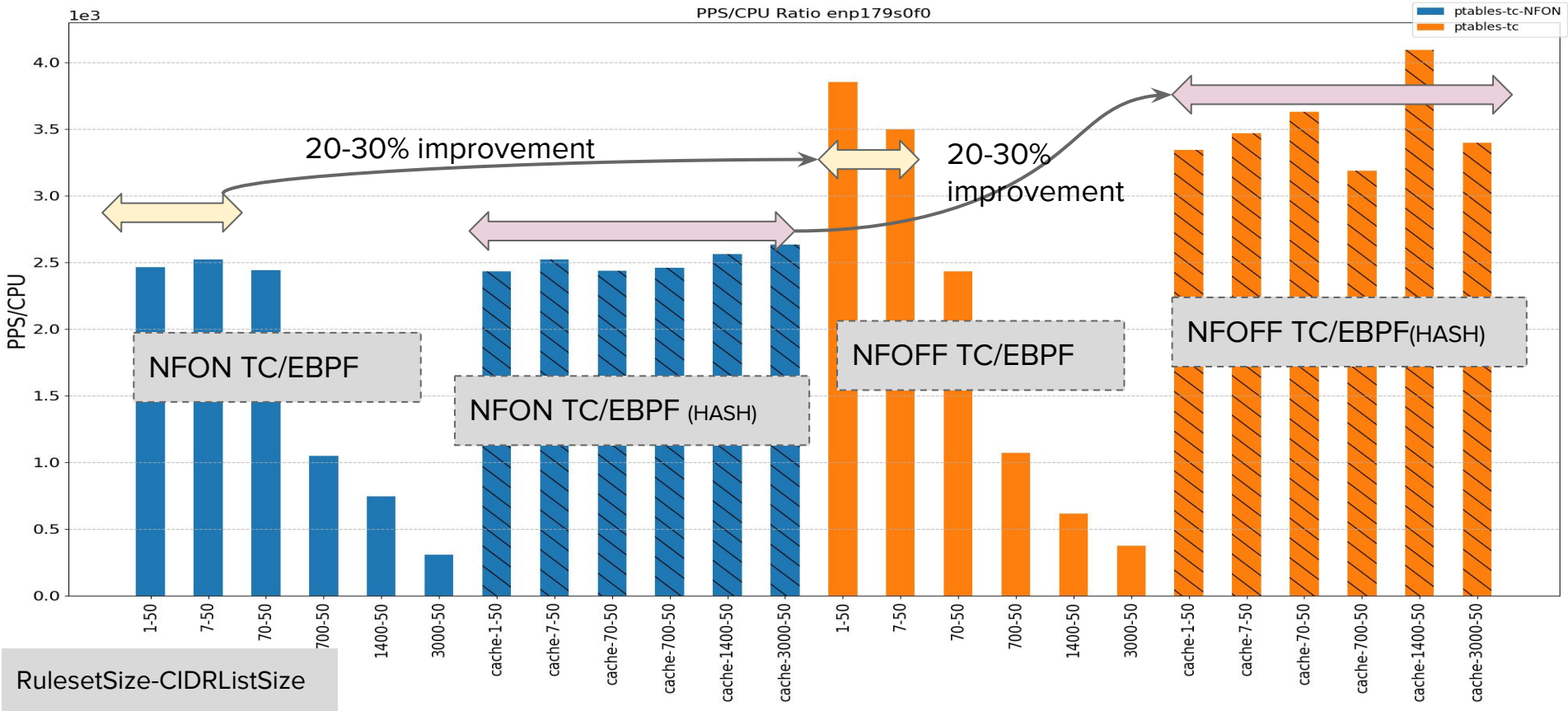


Ingress Host NFON Summary

- Ruleset Linear iteration as expected got worse with rules
  - TC/XDP ebpf did better than IPtables/IPSet
  - IPtables/IPSet results after 700 ruleset intolerable
  - No difference between XDP and TC because packets go to stack
- Caching improved the performance of linear search only for TC/XDP
  - Not much of an improvement for Conntracking
  - Ebpf both at TC and XDP with caching outperformed conntracking

Host Ingress NF compiled out of Kernel

Host Ingress TC NF OFF vs NF ON PPS/CPU

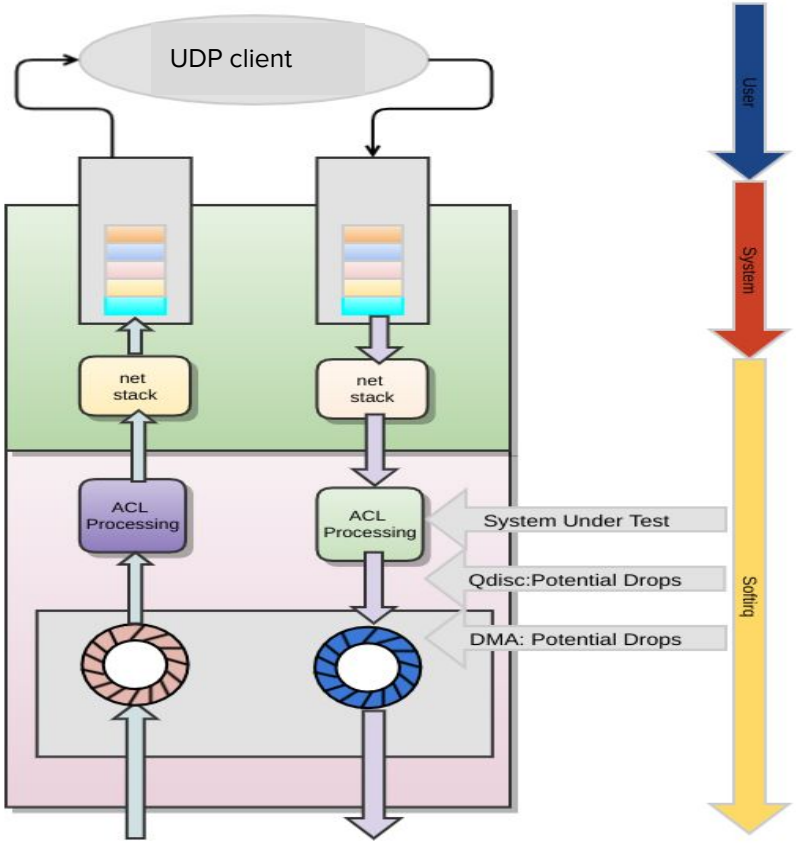


RulesetSize-CIDRListSize

Ingress Host NF OFF Summary

- Ruleset Linear iteration as expected got worse with rules
- Caching improved the performance of linear search
- Compared to NFON
  - Improves performance by about 20-30%
  - Lowered CPU utilization and latency for same packet rate

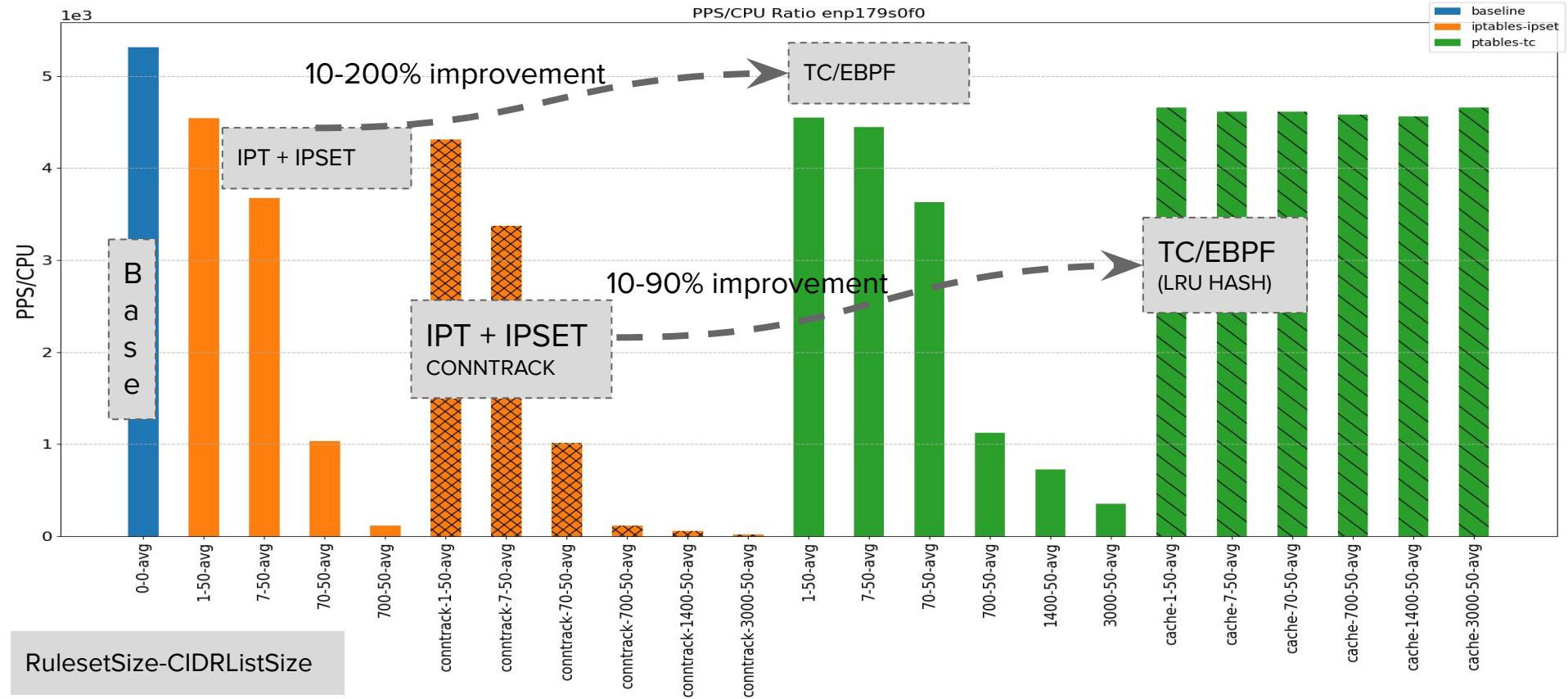
Host: Egress Processing



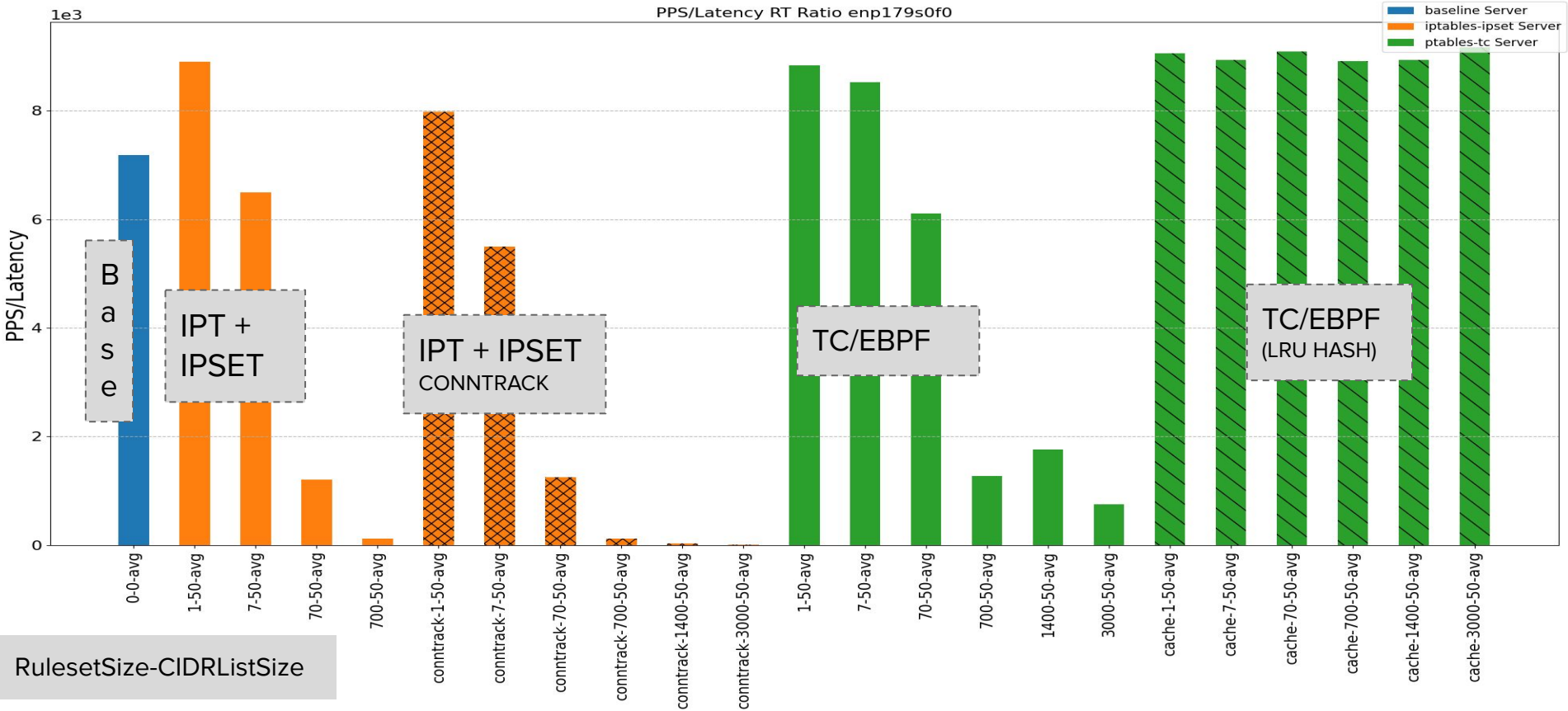
- NIC irq tied to one core
- Single threaded UDP client tied to second core
  - Mostly only one core used
- Sending rate set to MLFRR
  - Drops [0, 0.0001%]
- Capture:
    - Throughput
    - latency
    - CPU breakdown
    - Drops

Host Egress NF compiled Into Kernel

Host: Egress NF ON PPS/CPU (Higher Better)



Host: Egress NF ON Latency/CPU (Higher Better)

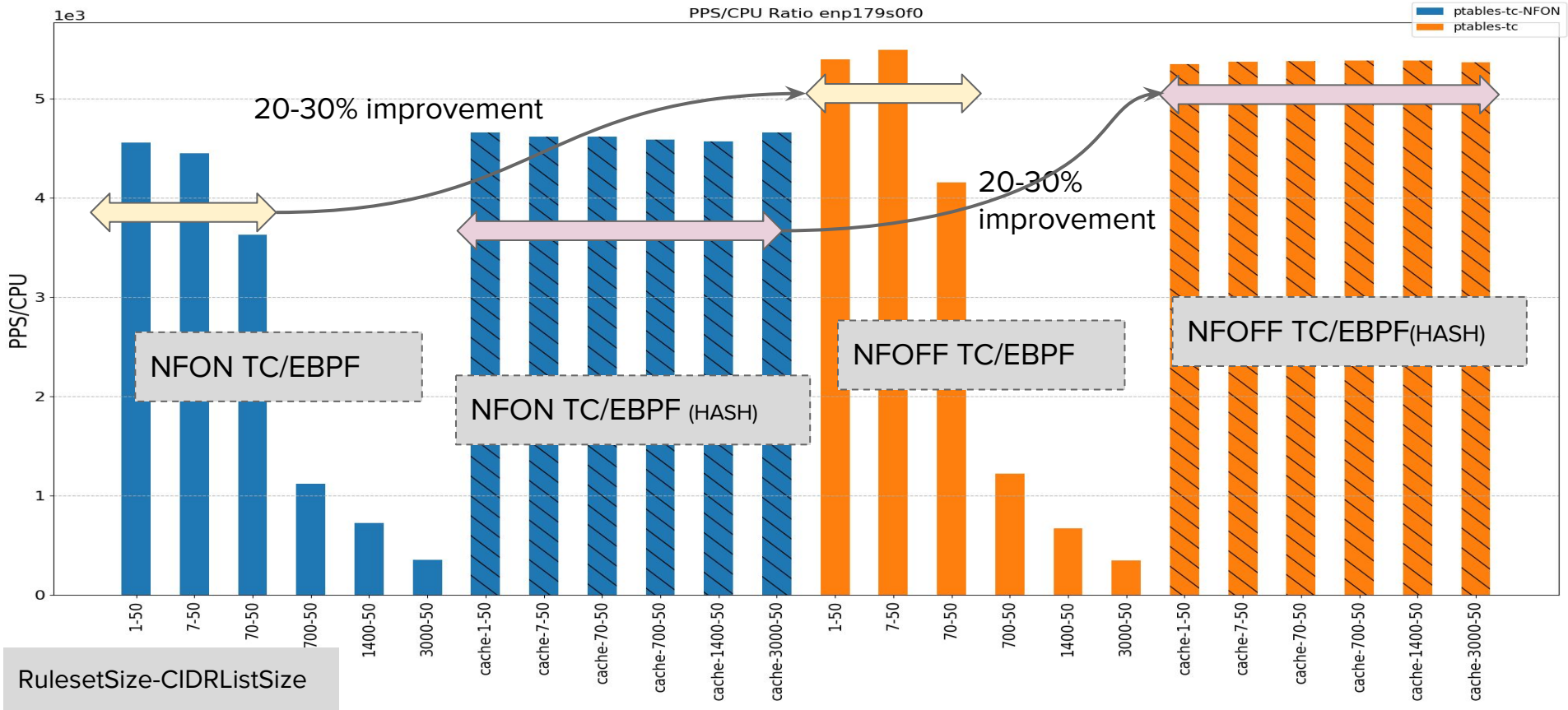


Egress Host NF ON Summary

- Ruleset Linear iteration as expected got worse with rules
  - TC/XDP ebpf did better than IPtables/IPSet
  - IPtables/IPSet results after 700 ruleset intolerable
  - No difference between XDP and TC because packets go to stack
- Caching improved the performance of linear search only for TC/XDP
  - Very little or no improvement for IPtables/IPSet
  - Ebpf both at TC and XDP with caching outperformed conntracking

Host Egress NF compiled out of Kernel

Host Egress TC NF OFF vs NF ON PPS/CPU



Egress Host NF OFF Summary

- Ruleset Linear iteration as expected got worse with rules
- Caching improved the performance of linear search
- Compared to NFON
  - Improves performance by about 20-30%
  - Lowered CPU utilization and latency for same packet rate

Conclusion

- EBPF provides better performance than the trio of IPTables/IPSet/Conntracking
  - Both in the datapath and control path
- Caching helps in all cases
- Compiling out Netfilter improves EBPF performance as normalized over CPU and latency by 20-30% in all workloads

Acknowledgements

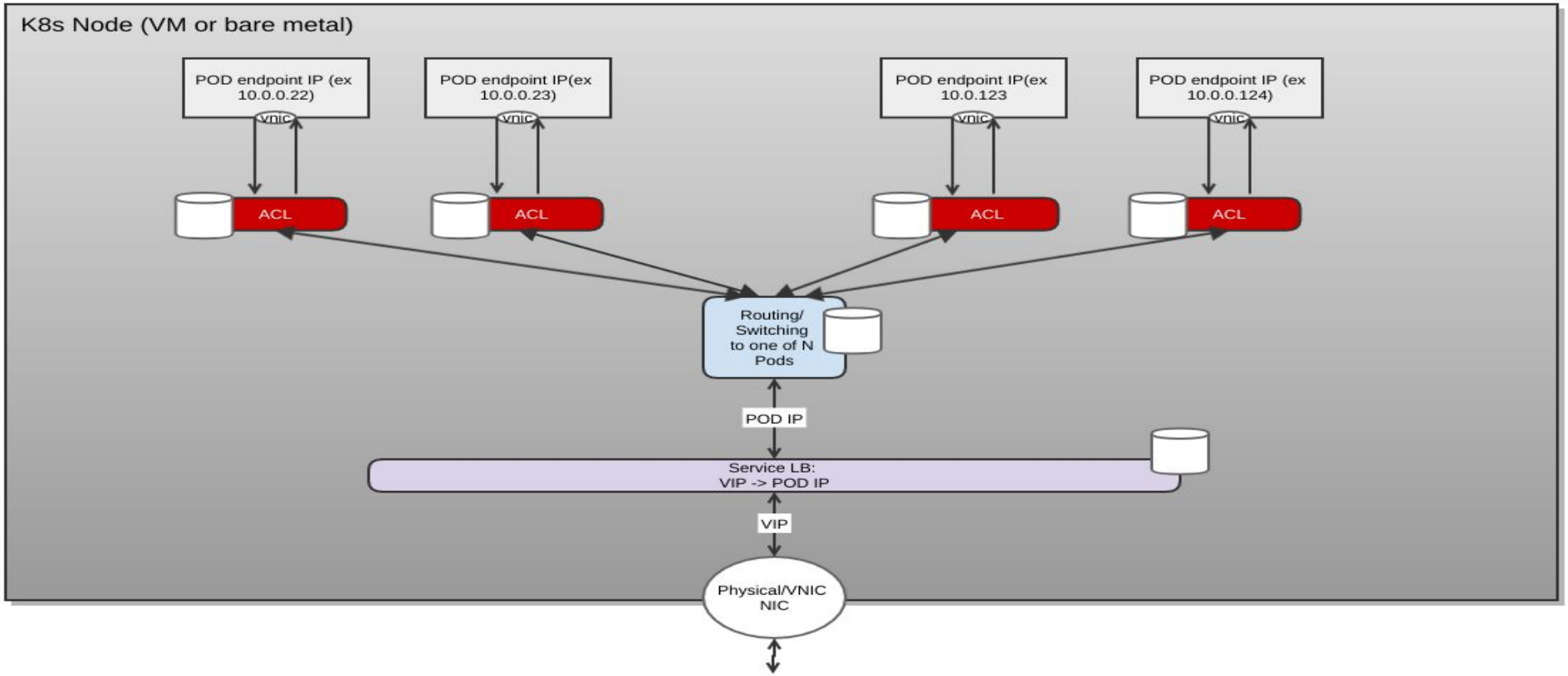
- Paul Mininni
- Pedro Tammela

Post Mortem

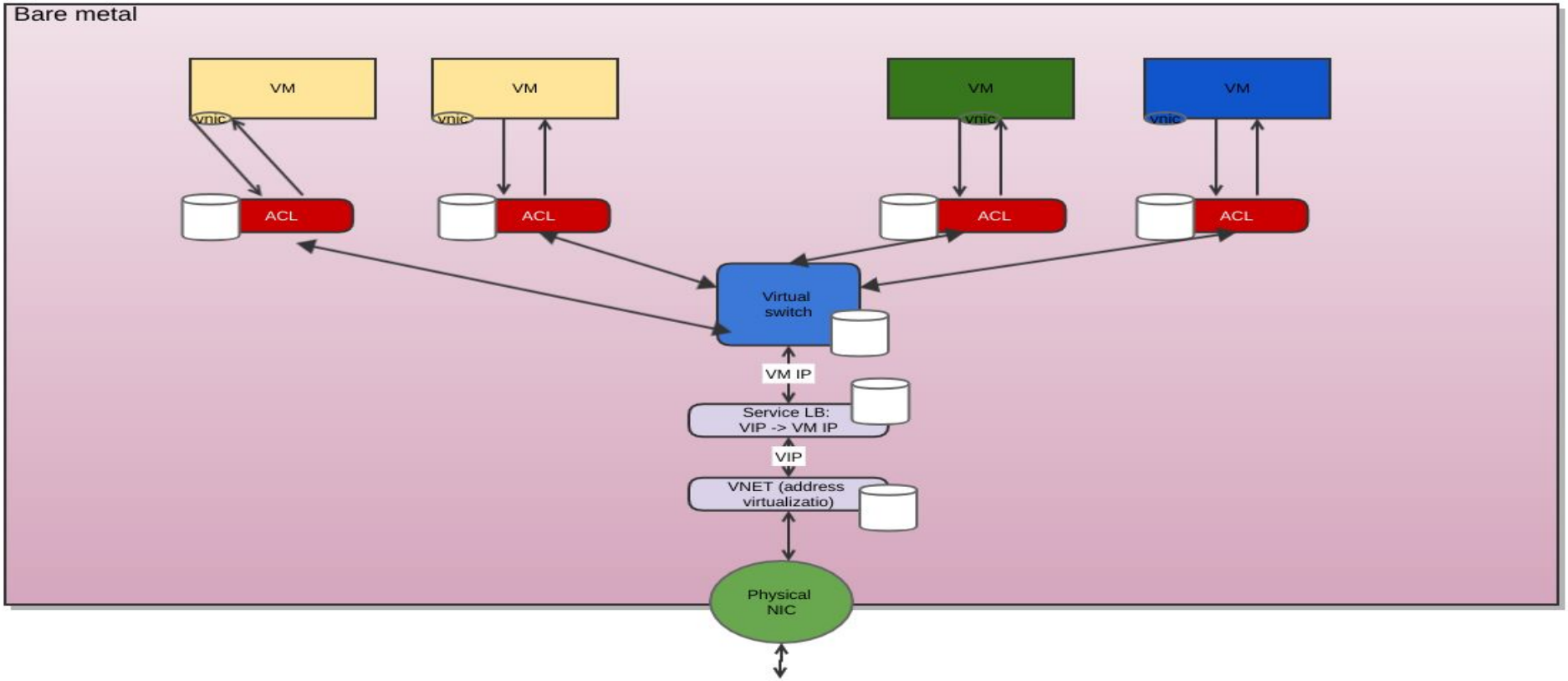
- Kudos to Florian Westphal(netfilter maintainer) who fixed the performance degradation shown in NFON vs NFOFF after attending the talk at netdev 0x15
  - https://git.kernel.org/pub/scm/linux/kernel/git/netdev/net-next.git/commit/?id=fdacd57c79b79a03c7ca88f706ad9fb7b46831c1
  - We gained back the 20-30% we showed with and without NF being compiled in
  - There's still a ~1% degradation if you run "iptables -L" (even when no rules are a loaded)
    - Still a huge improvement!
    - We thank Florian!

Q &A

ACL Sample Space: Kubernetes



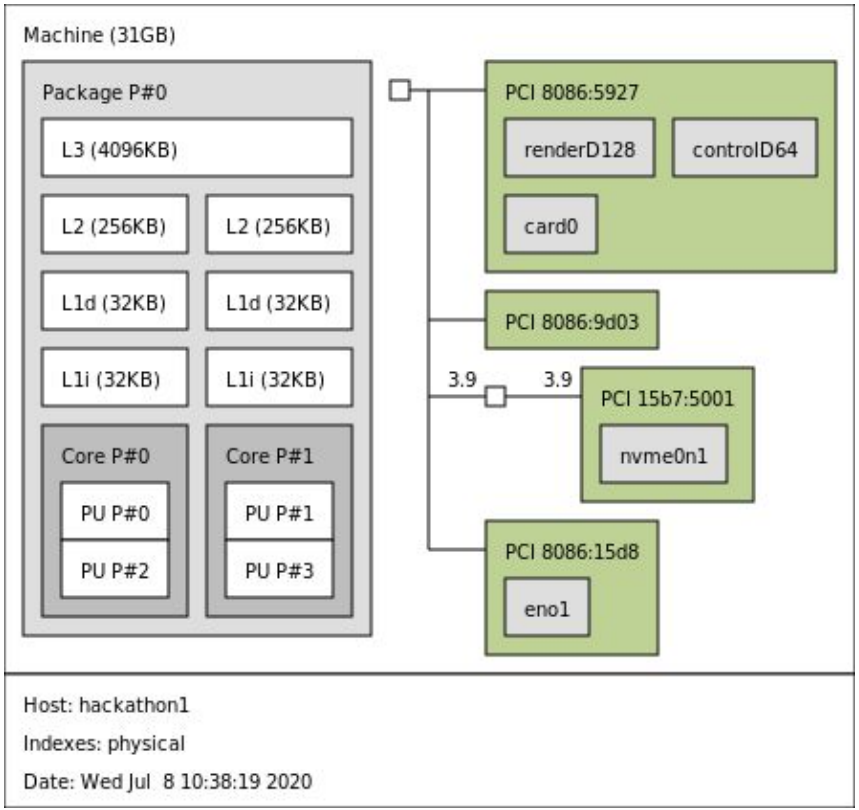
ACL Sample Space: MS Azure VFP



Back Slides

Control Path Testing

Control Testing: Hardware Setup

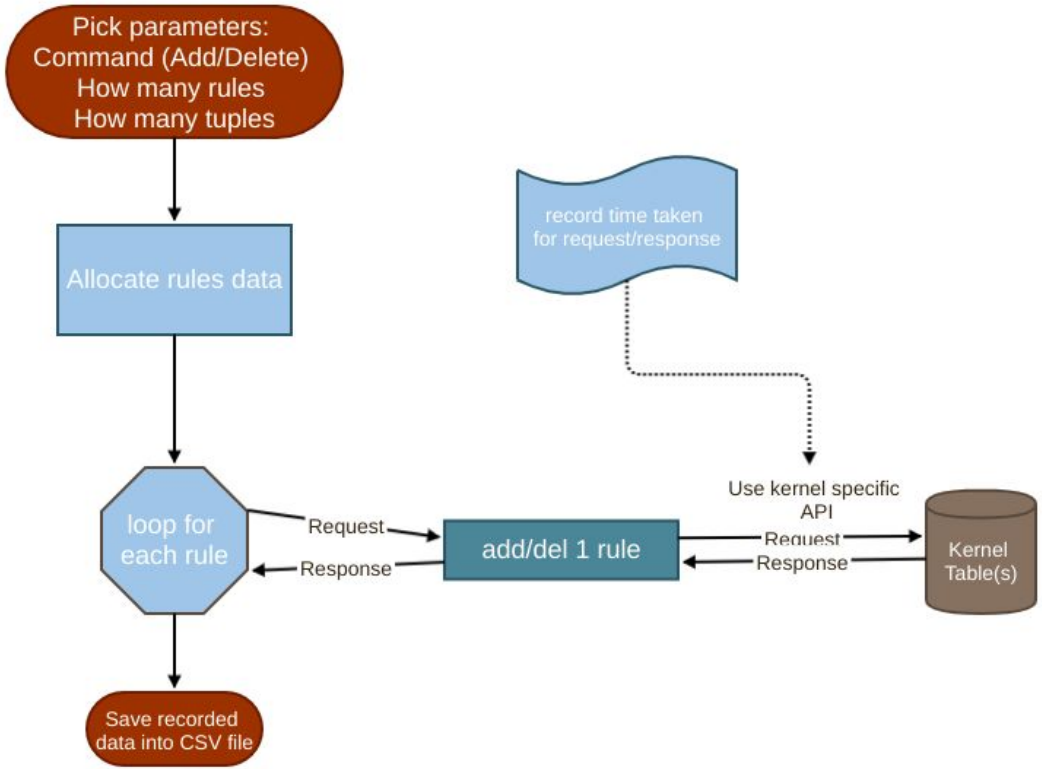


- Intel NUC is equipped with 7th Generation Intel Core i7-7567U 3.4Ghz
- Dual channel DDR4-2133 SODIMMs 32GB.
- Hyperthread off
  - meaning “Core P#0” and “Core P#1” of will each have one of the threads off

Control Path Tests

- Why is this important?
  - The faster you can update the datapath the sooner the rules take effect
  - Consider scenario where resource lifetimes are short, e.g., K8s Pods
    - How fast you update determines your capacity (Pods/Sec provisioned)
- Two metrics considered
  - Latency cost of adding/deleting the Nth+1 ACL rule
  - Latency cost of adding/deleting N ACL rules (one at a time, not batching)

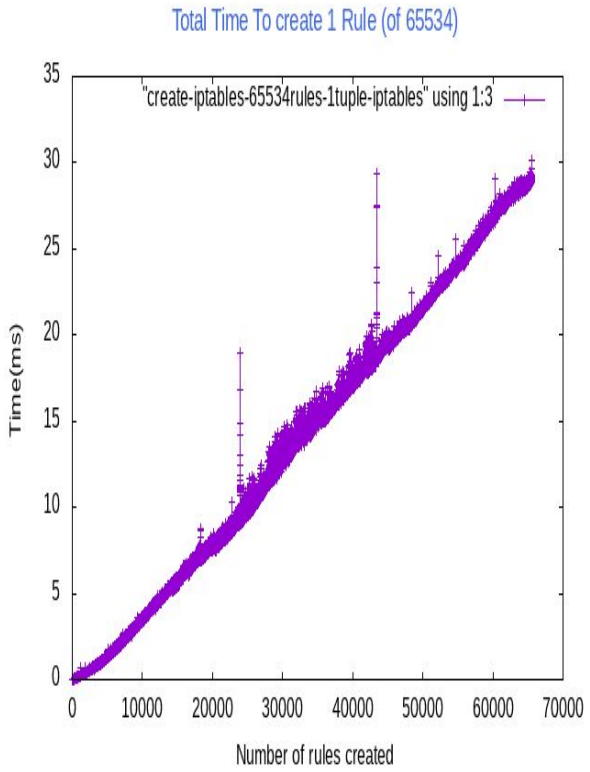
Control Testing: Software Infrastructure

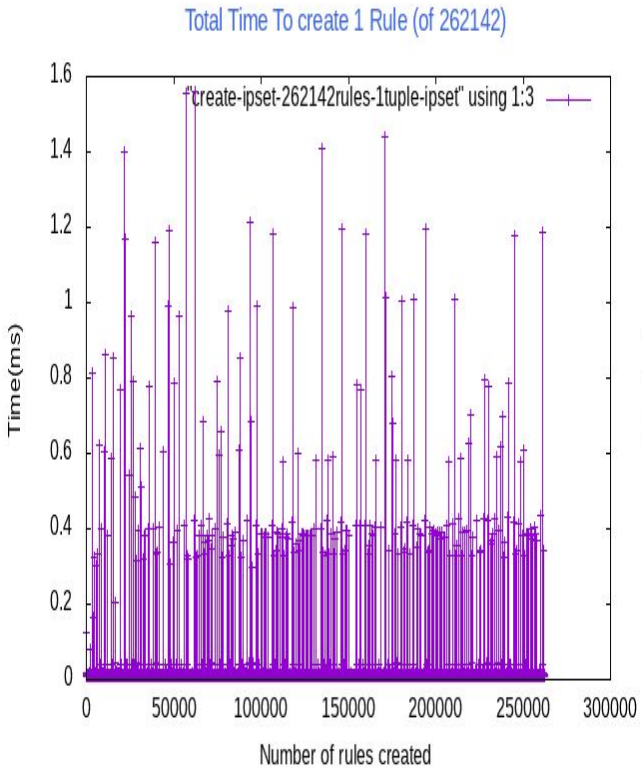


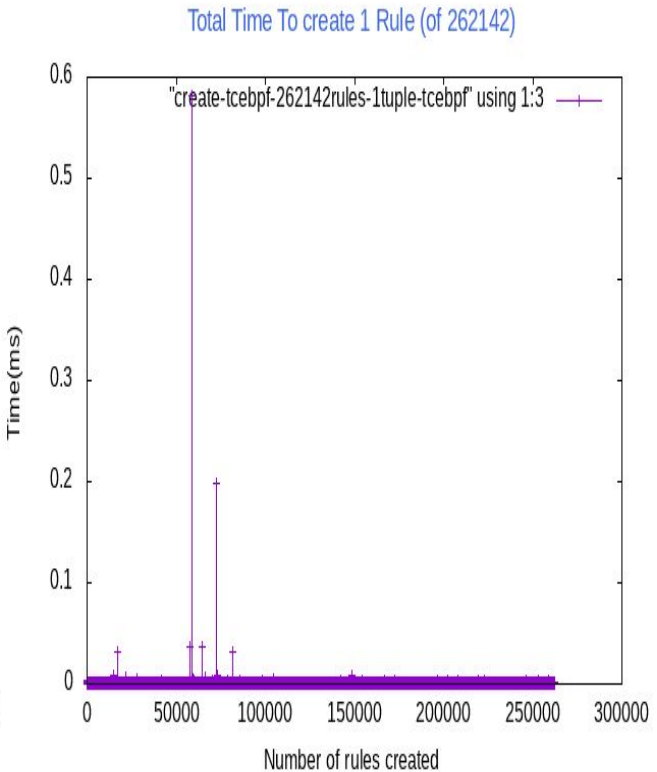
Wrote control programs for:

1. IPtables/IPSet
2. TC/Ebpf
3. XDP/Ebpf
4. Flower

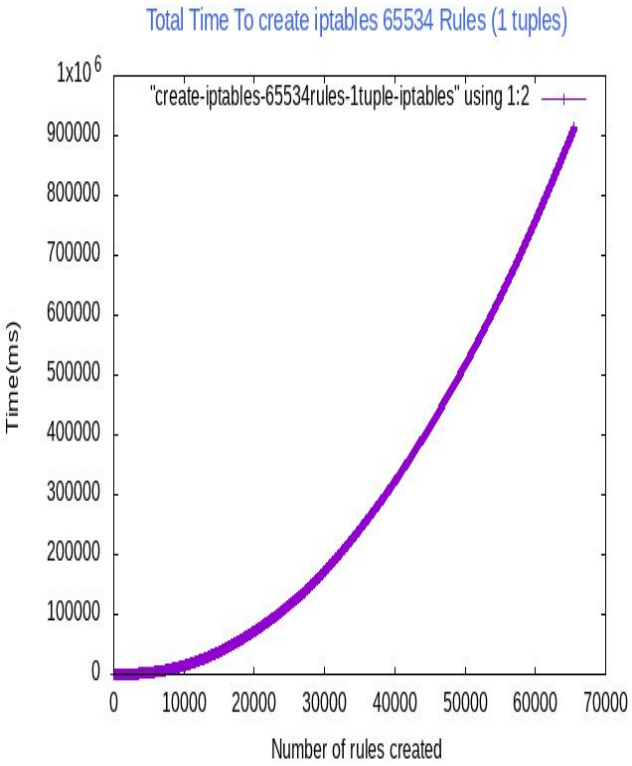
Control: Add Latency Nth+1 Rule

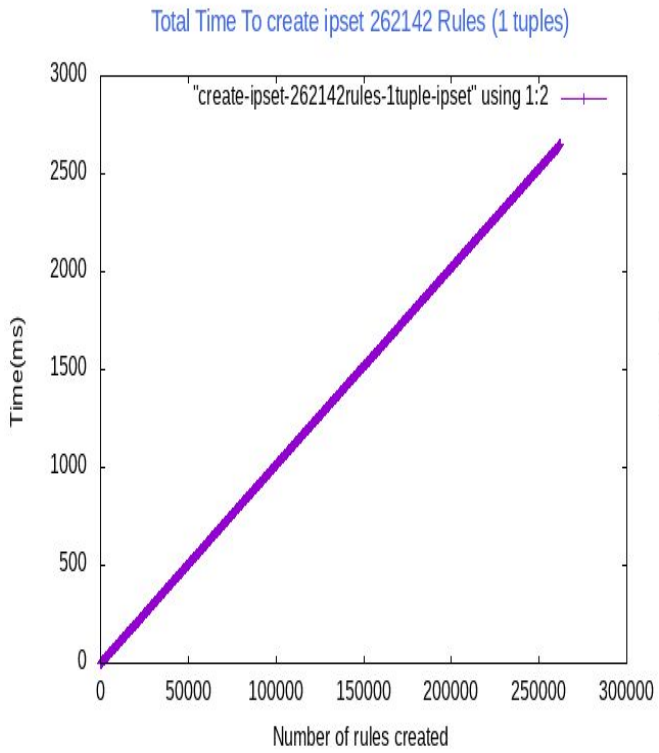


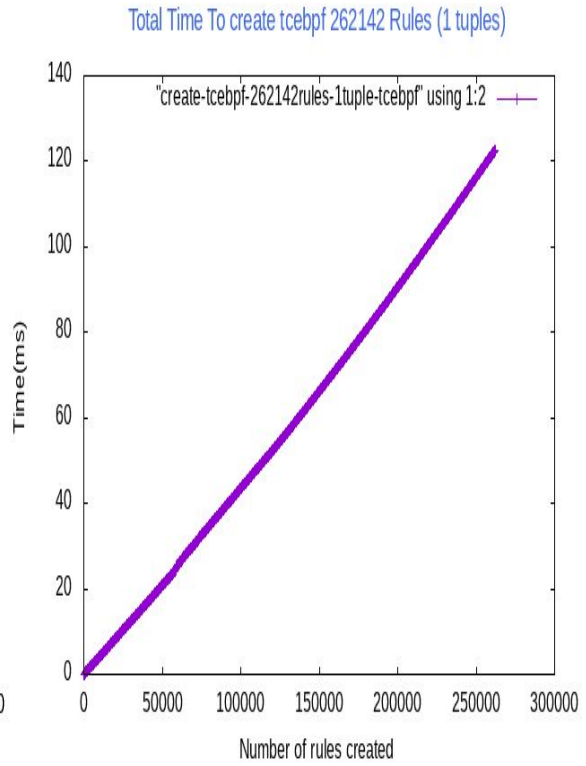




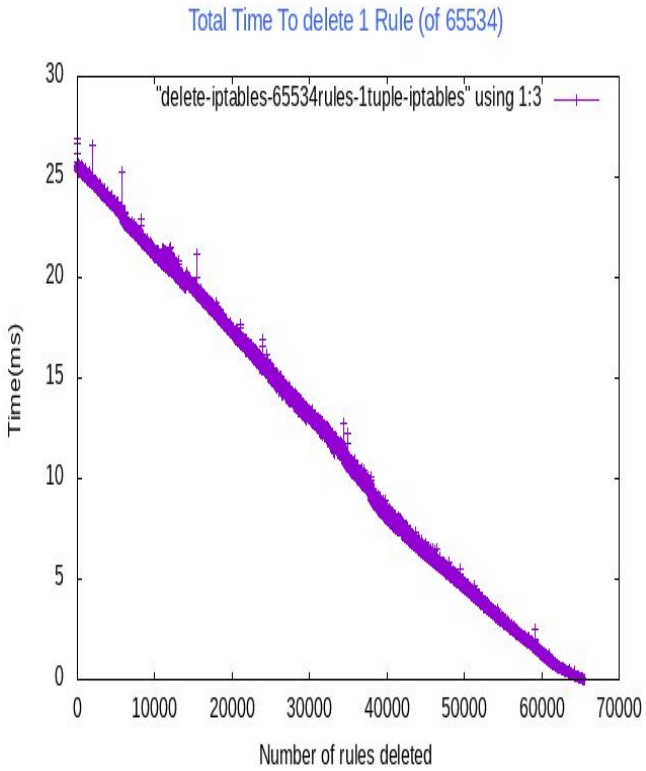
Control: Latency For Adding N Rules

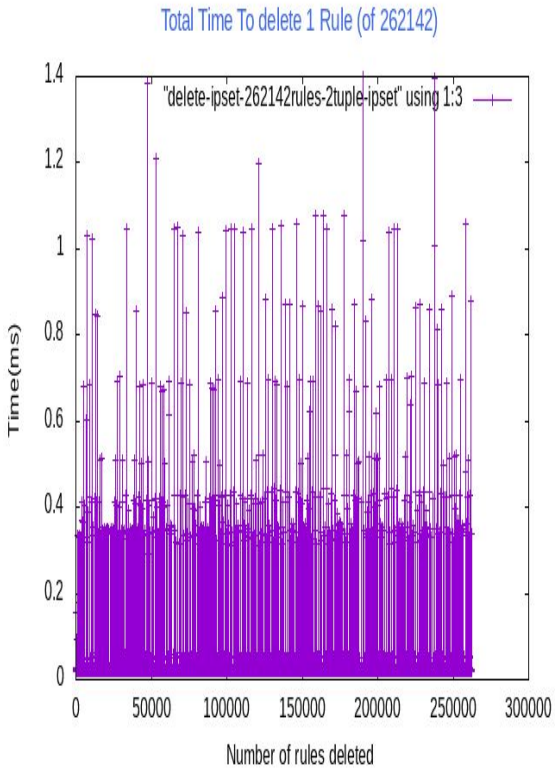


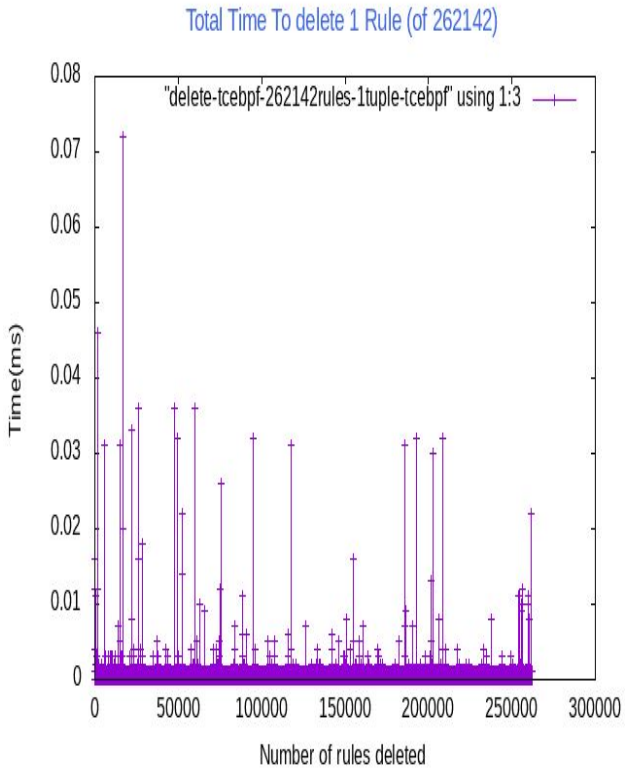




Control: Latency Deleting Nth Rule

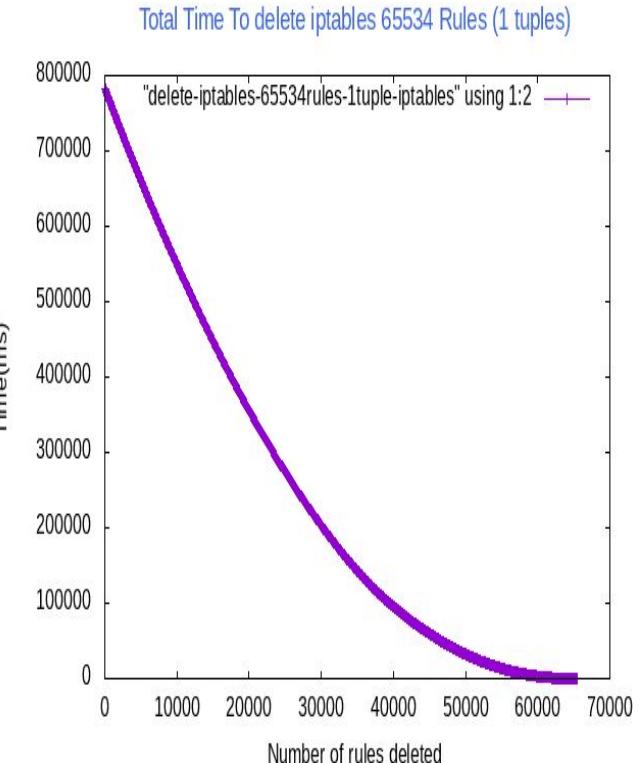


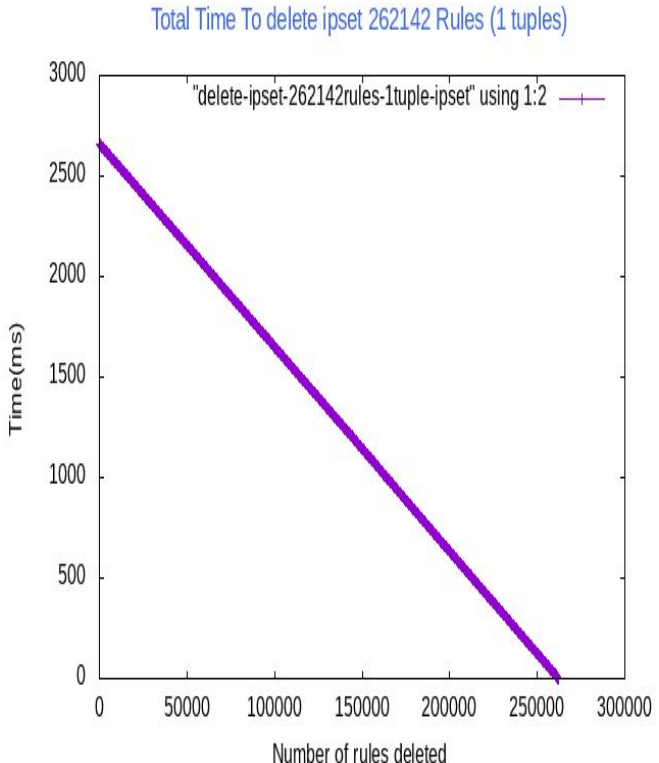


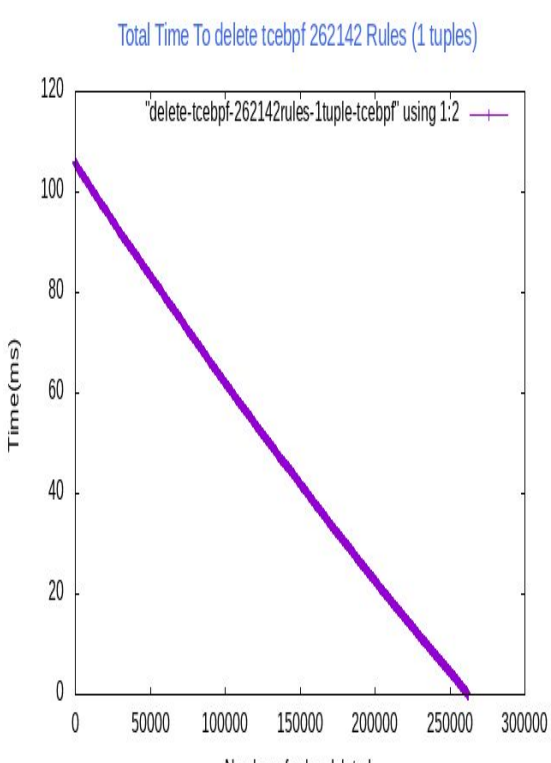


Control: Latency For Deleting N Rules

Incrementally 1 Rule at a time







Control Path Results

- Iptables Nth + 1 rule add/del is linear in time. IPset and ebpf are constant
- For a total of N Rules:
  - Quadratic complexity for Iptables
  - Linear for IPSet and ebpf
- Throughput:
  - Ebpf is an order of magnitude better in throughput vs IPset
  - IPSet several times better than plain iptables in throughput
- CPU Consumption
  - Iptables is the worst and IPSet is the best
    - IPSet uses netlink which is asynchronous where ebpf is synchronous/blocking

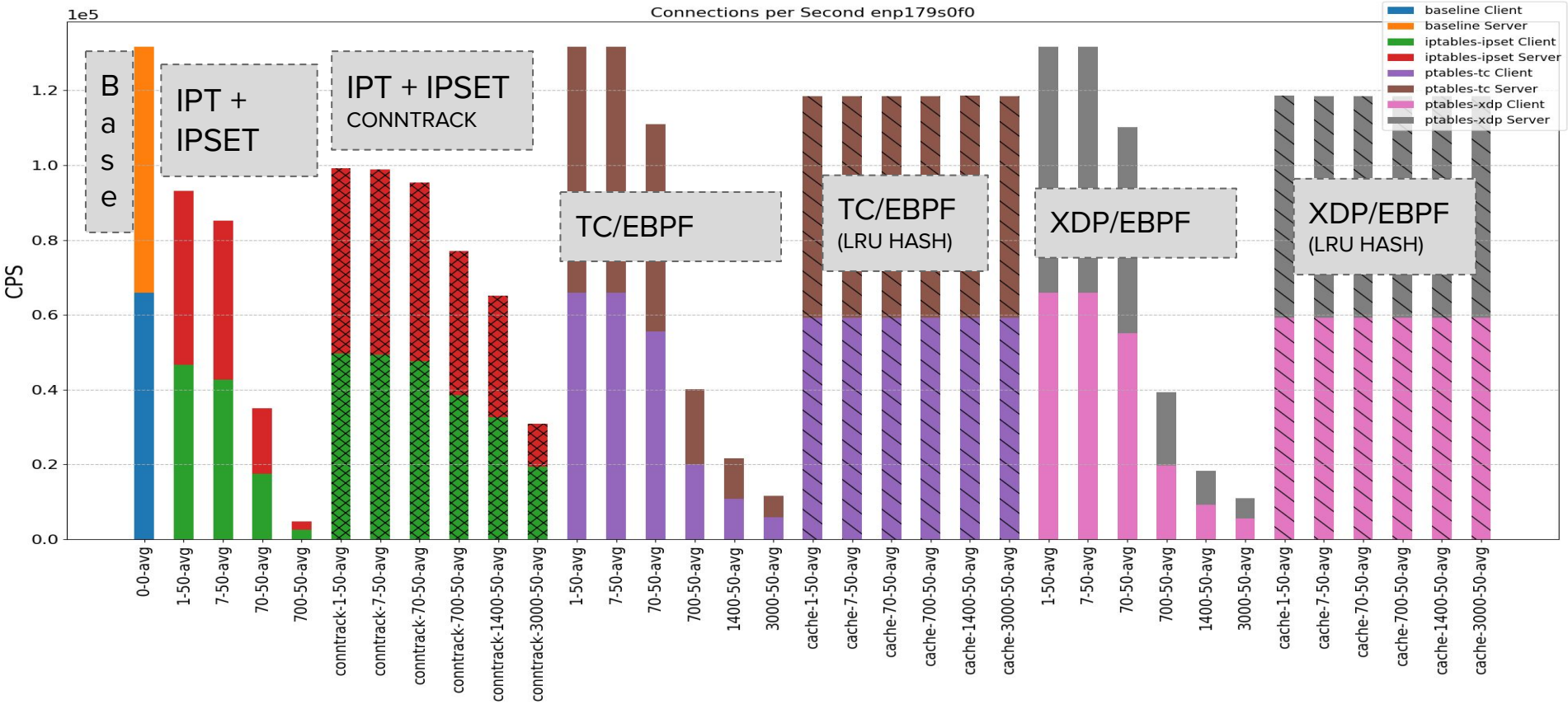
Iptables Control: Add 1 Rule to 32K

```
socket(AF_INET, SOCK_RAW, IPPROTO_RAW)  = 3
fcntl(3, F_SETFD, FD_CLOEXEC)           = 0
getsockopt(3, SOL_IP, IPT_SO_GET_INFO, "filter\0..\0\0\0\0\0\0\0\0\0\0\0\0"..., [84]) = 0
brk(0x55aef6e3a000)                     = 0x55aef6e3a000
getsockopt(3, SOL_IP, IPT_SO_GET_ENTRIES, "filter\0\0..0\0\0"..., [4981656]) = 0
brk(0x55aef72fa000)                     = 0x55aef72fa000
setsockopt(3, SOL_IP, IPT_SO_SET_REPLACE, "filter\0\0\...0\0\0\0\0\0\0\0"..., 4981864) = 0
setsockopt(3, SOL_IP, IPT_SO_SET_ADD_COUNTERS, "filter\0\0\0\0\0\0..\0\0\0\0"..., 524392) = 0
close(3)                                = 0
```

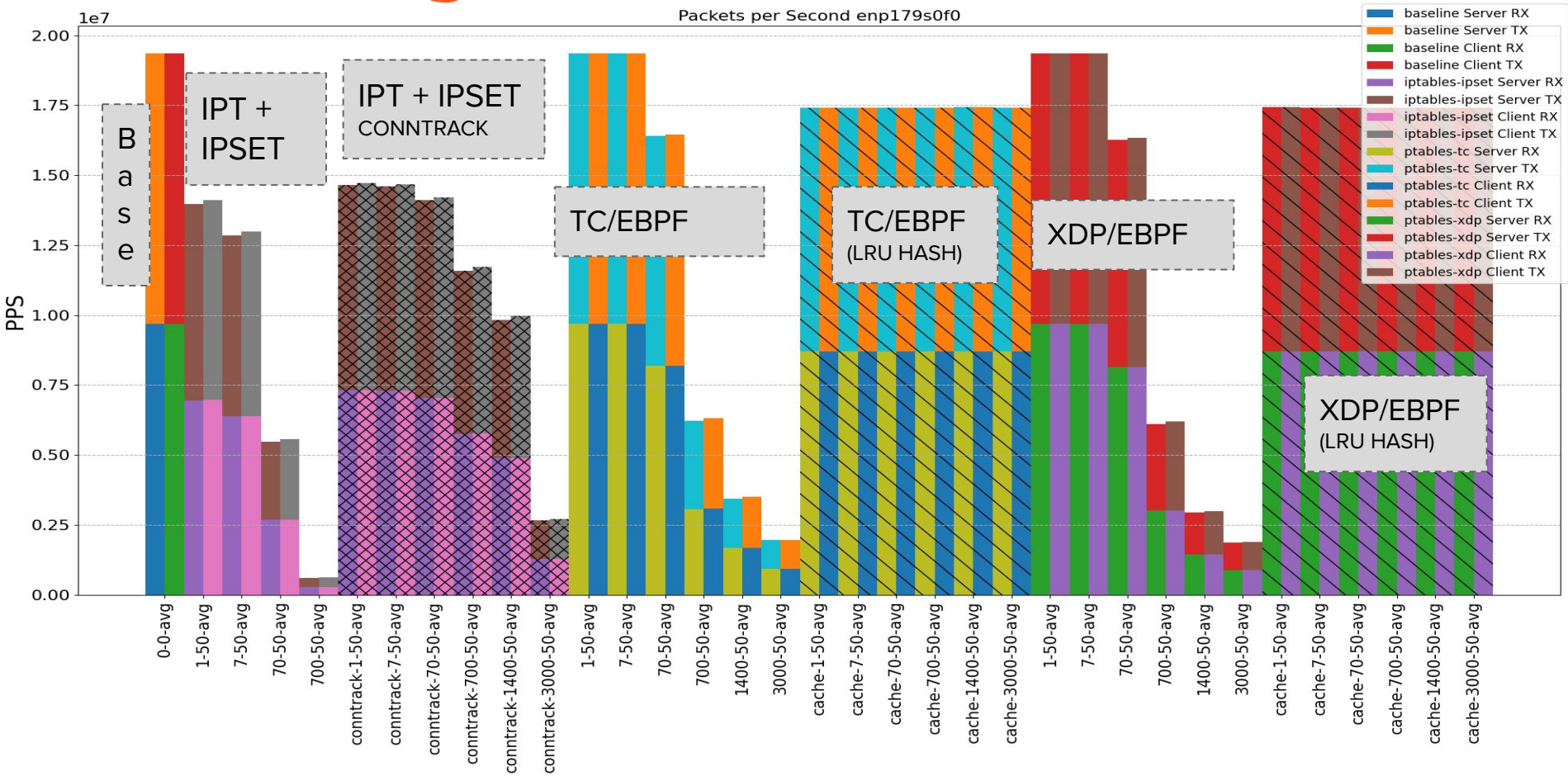
Add/DEL/mod of 1 ruleset requires copying all rulesets to and from userspace

Forwarding: NFON

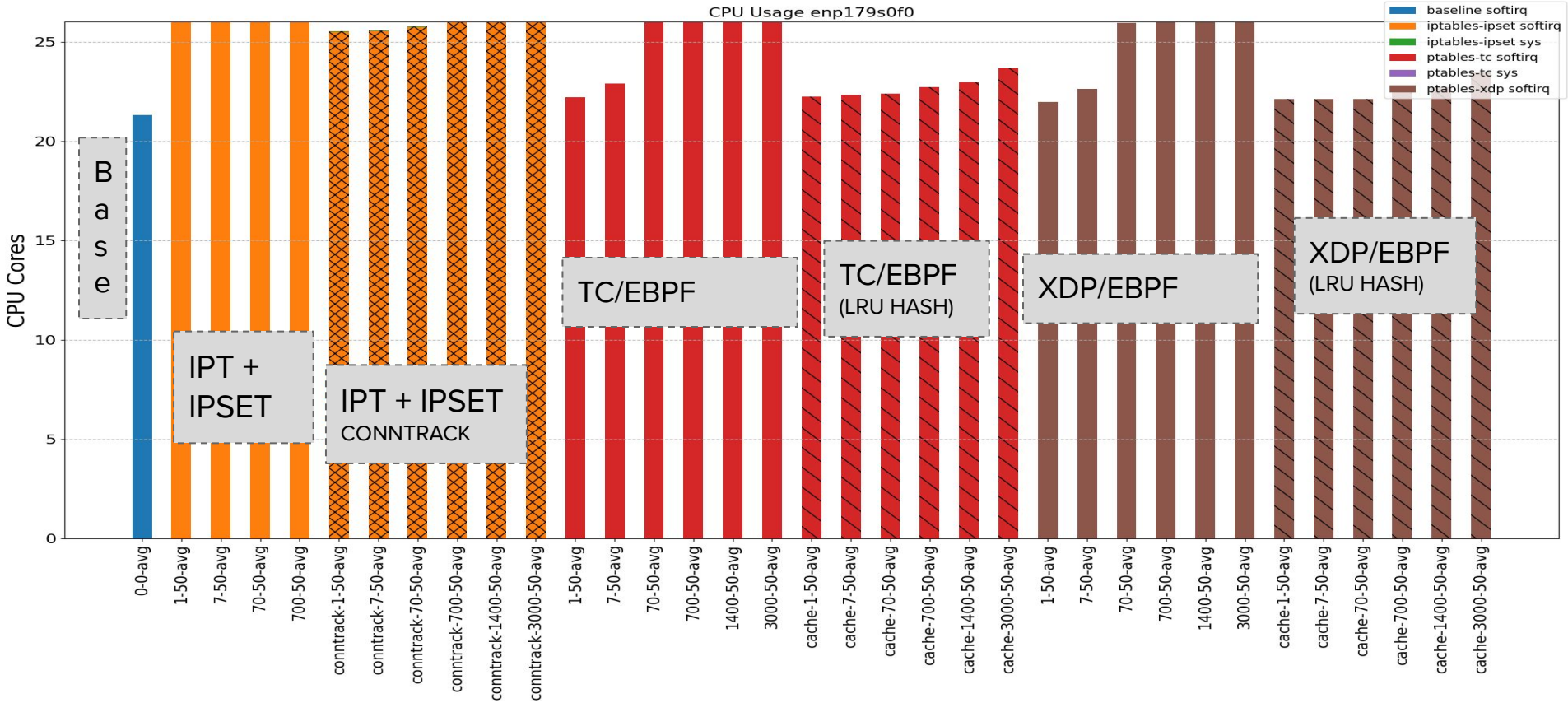
Forwarding NFON: CPS (Higher Better)



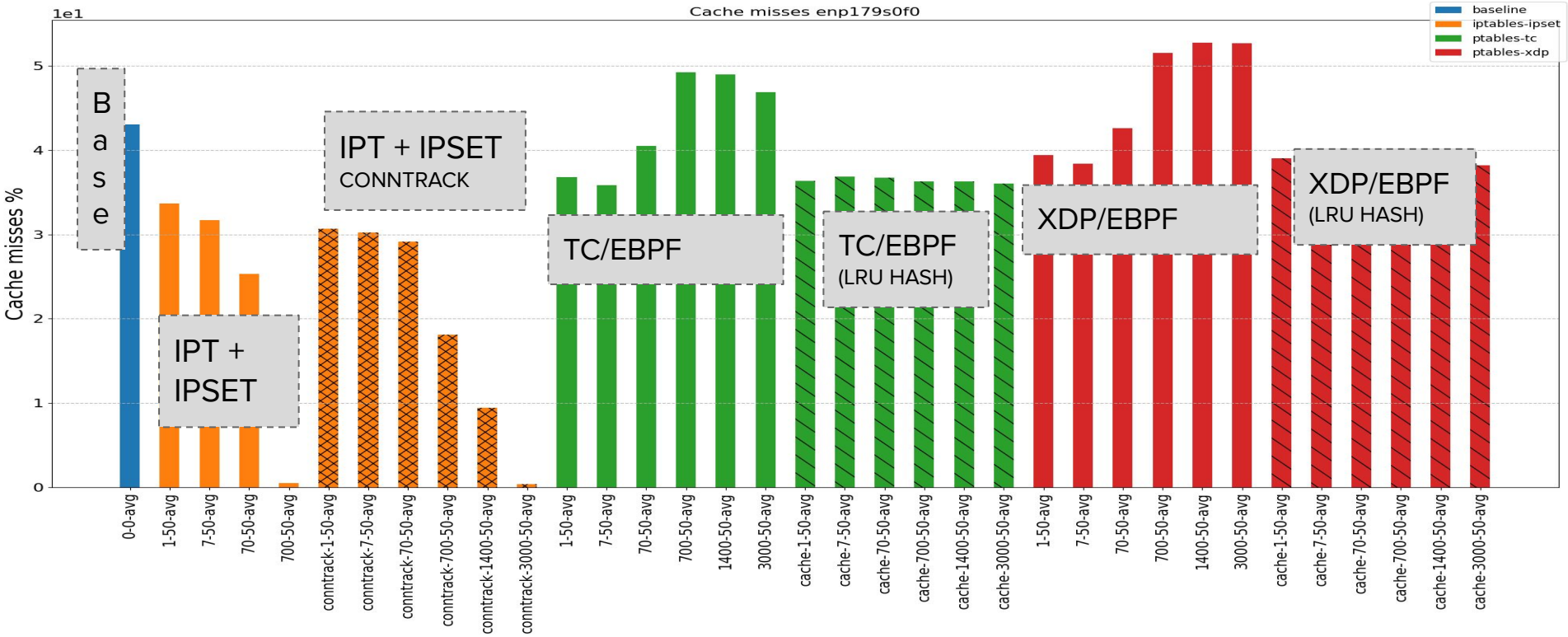
Forwarding NFON: PPS (Higher Better)



Forwarding NFON: CPU (Lower Better)

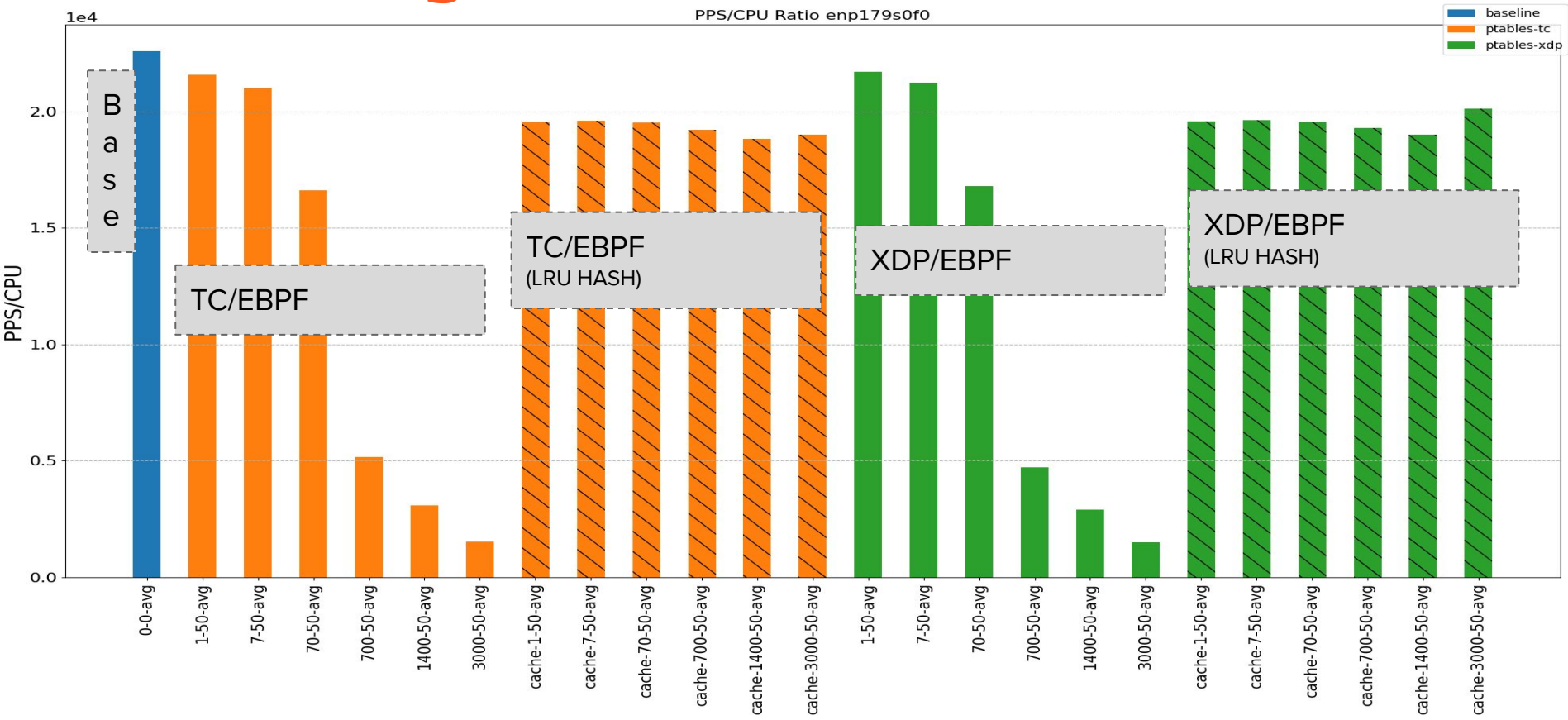


Forwarding NFON: CPU Cache Misses (Lower Better)

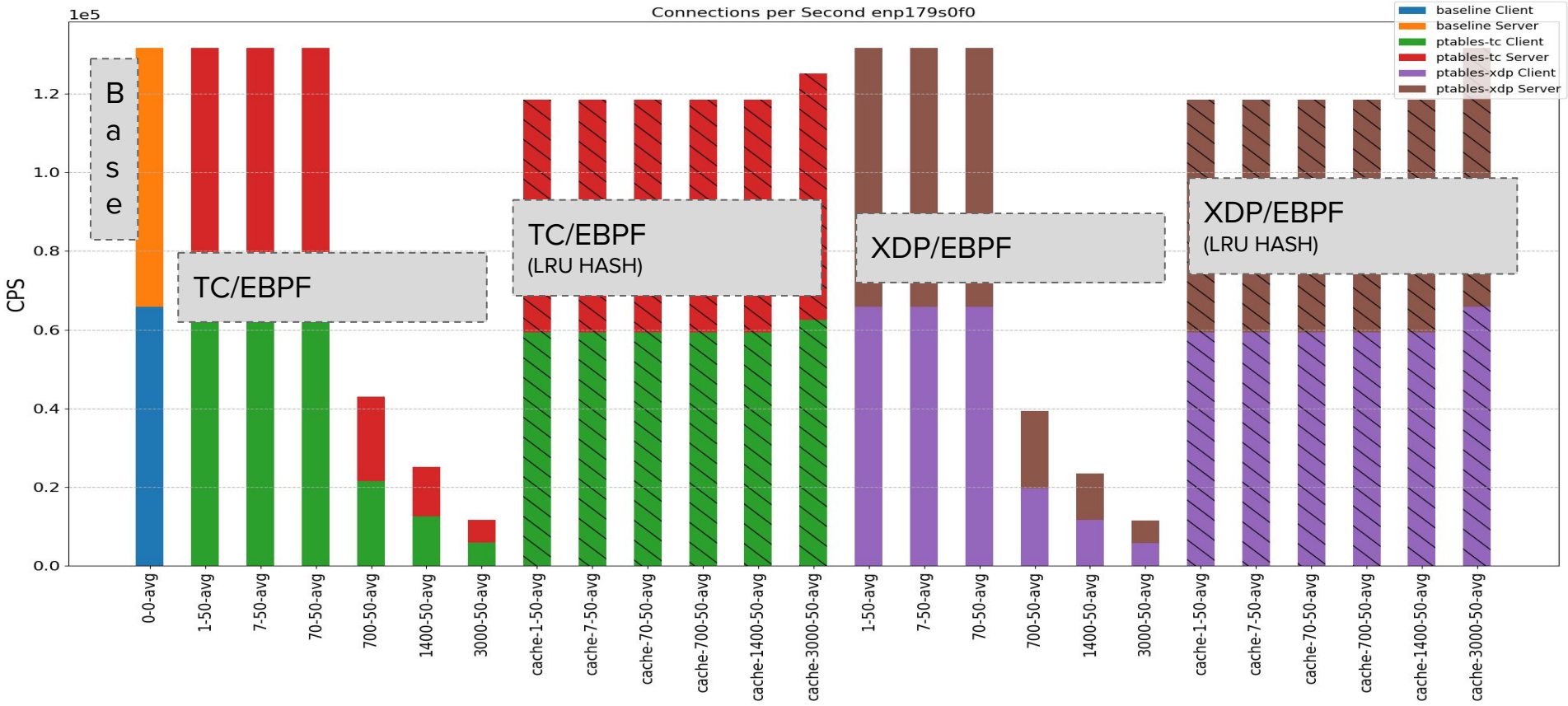


Forwarding NF compiled out of Kernel

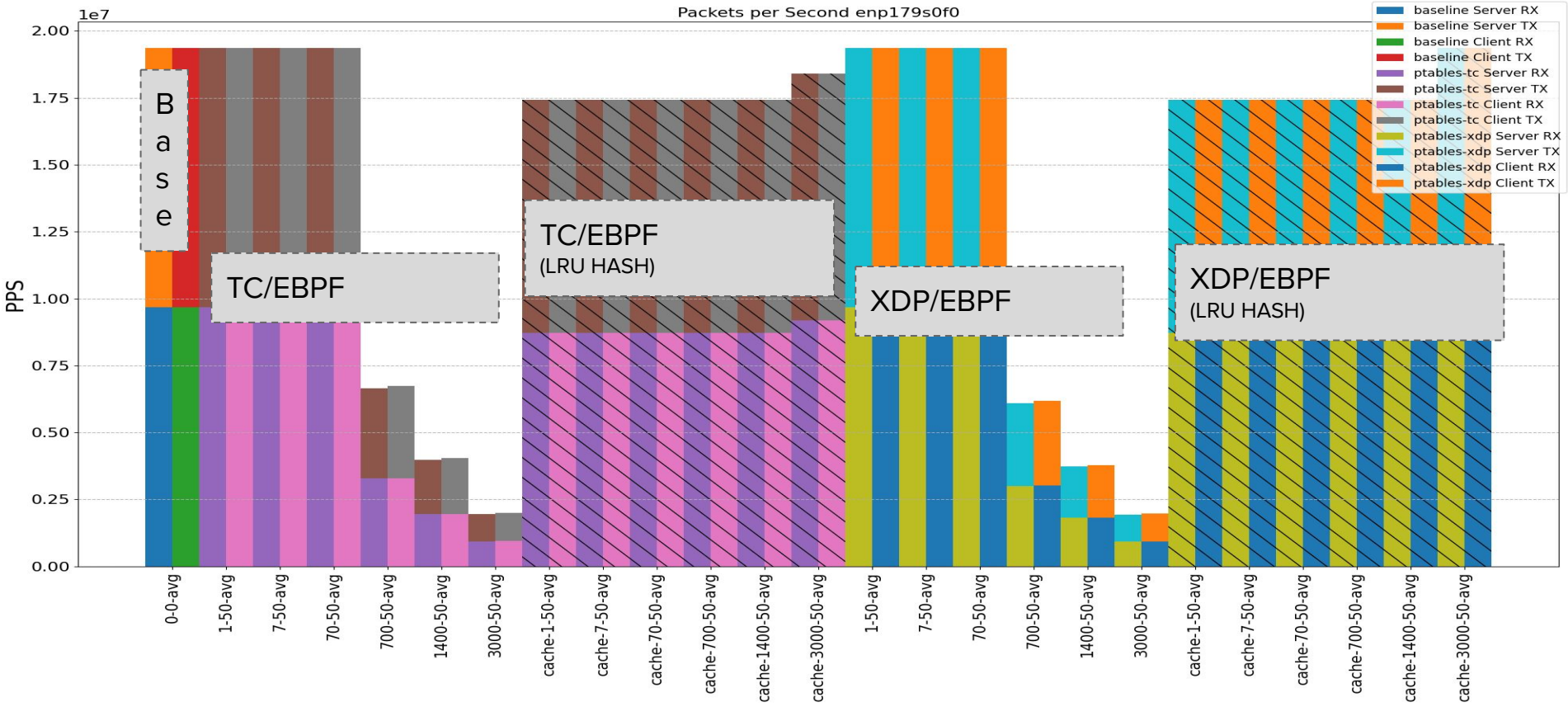
Forwarding NFOFF: PPS/CPU (Higher Better)



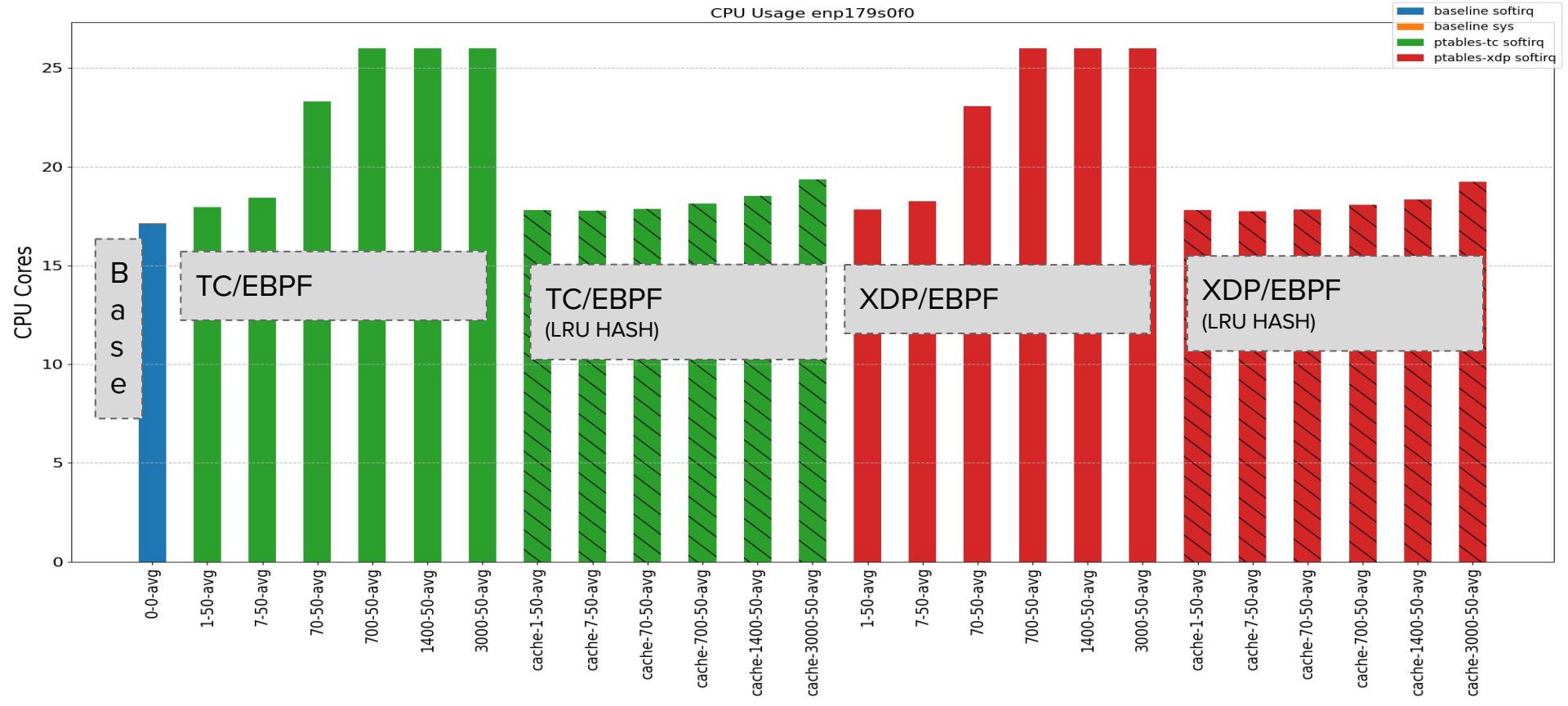
Forwarding NFOFF: CPS (Higher Better)



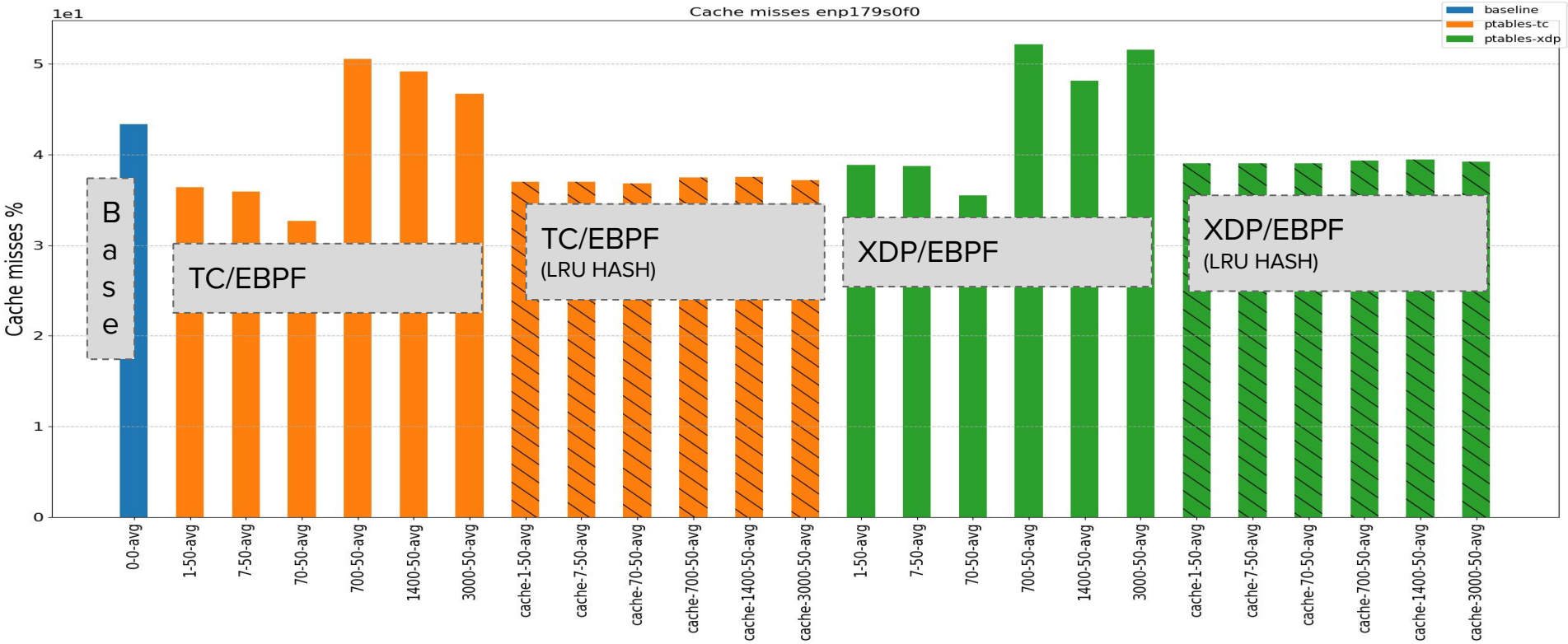
Forwarding NFOFF: PPS (Higher Better)



Forwarding NFOFF: CPU (Lower Better)



Forwarding NFOFF: CPU Cache Miss (Lower Better)



Challenges With IPTables Datapath

NFON: TC/Ebpf 70 Rulesets Perf Output

```
 22.61%  ksoftirqd/1  bpf_prog_5baaa2b0c31f333f  [k] bpf_prog_5baaa2b0c31f333f
 13.43%  ksoftirqd/1  [kernel.kallsyms]          [k] ipt_do_table
 10.61%  ksoftirqd/1  [kernel.kallsyms]          [k] _raw_spin_lock
  3.79%  ksoftirqd/1  [i40e]                     [k] i40e_lan_xmit_frame
  3.48%  ksoftirqd/1  [kernel.kallsyms]          [k] fib_table_lookup
  3.23%  ksoftirqd/1  [i40e]                     [k] i40e_napi_poll
  1.88%  ksoftirqd/1  [kernel.kallsyms]          [k] fib_lookup_good_nhc
  1.87%  ksoftirqd/1  [kernel.kallsyms]          [k] __netif_receive_skb_core
  1.57%  ksoftirqd/1  [kernel.kallsyms]          [k] nf_hook_slow
  1.53%  ksoftirqd/1  [kernel.kallsyms]          [k] ip_route_input_slow
  1.49%  ksoftirqd/1  [kernel.kallsyms]          [k] __dev_queue_xmit
  1.29%  ksoftirqd/1  [kernel.kallsyms]          [k] ip_forward
  1.17%  ksoftirqd/1  [kernel.kallsyms]          [k] __fib_validate_source
  1.16%  ksoftirqd/1  [kernel.kallsyms]          [k] kmem_cache_free_bulk
  1.12%  ksoftirqd/1  [kernel.kallsyms]          [k] eth_type_trans
  1.10%  ksoftirqd/1  [kernel.kallsyms]          [k] ip_finish_output2
  1.08%  ksoftirqd/1  [kernel.kallsyms]          [k] kmem_cache_alloc
  1.08%  ksoftirqd/1  [kernel.kallsyms]          [k] __build_skb
```

NFOFF: TC/Ebpf 70 Rulesets Perf Output

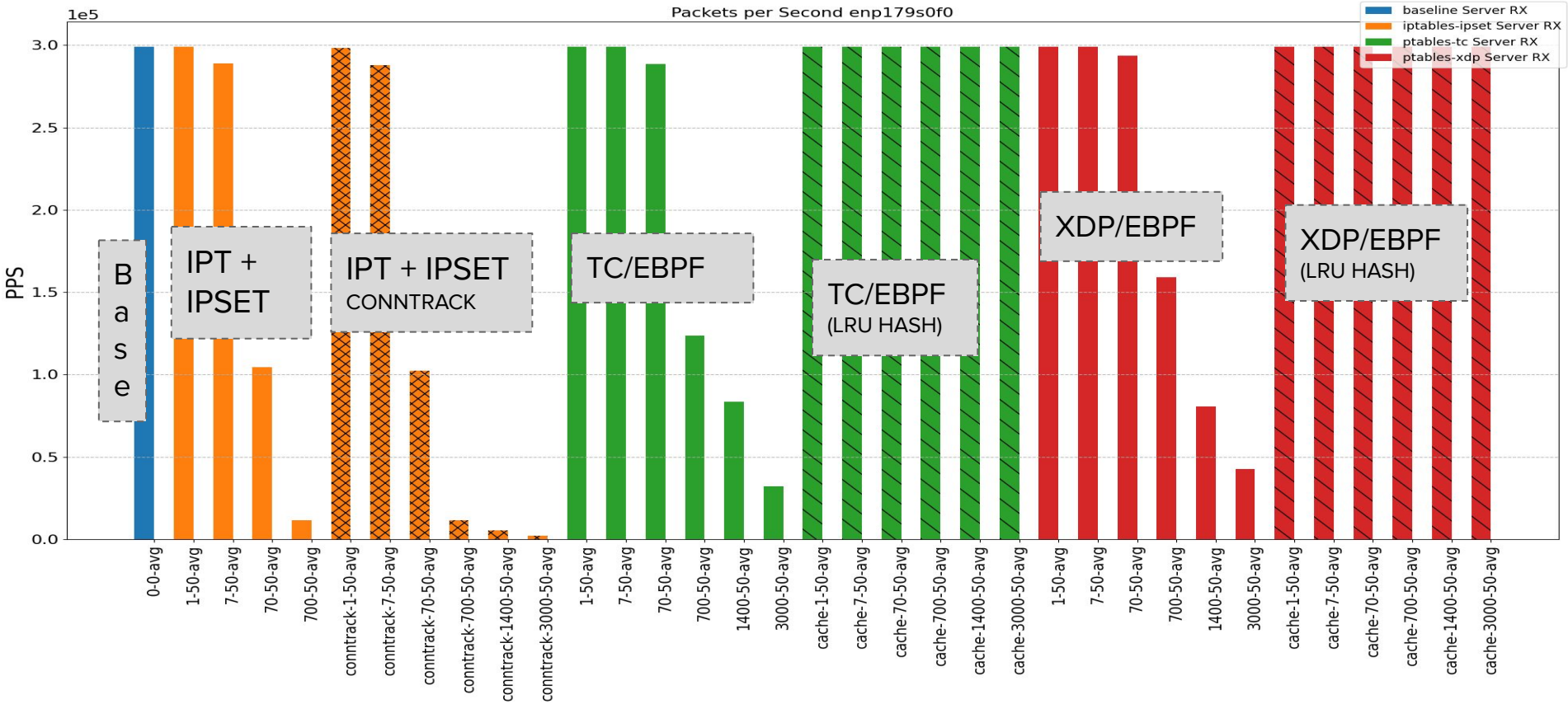
```
38.42%  ksoftirqd/1  bpf_prog_5baaa2b0c31f333f  [k] bpf_prog_5baaa2b0c31f333f
 2.88%  ksoftirqd/1  [kernel.kallsyms]          [k] _raw_spin_lock
 2.77%  ksoftirqd/1  [kernel.kallsyms]          [k] fib_table_lookup
 3.43%  ksoftirqd/1  [kernel.kallsyms]          [k] __netif_receive_skb_core
 2.36%  ksoftirqd/1  [i40e]                     [k] i40e_lan_xmit_frame
 1.73%  ksoftirqd/1  [kernel.kallsyms]          [k] ip_route_input_slow
 1.63%  ksoftirqd/1  [kernel.kallsyms]          [k] __dev_queue_xmit
 1.51%  ksoftirqd/1  [kernel.kallsyms]          [k] ip_forward
 1.51%  ksoftirqd/1  [i40e]                     [k] i40e_napi_poll
 1.41%  ksoftirqd/1  [kernel.kallsyms]          [k] fib_lookup_good_nhc
 1.35%  ksoftirqd/1  [kernel.kallsyms]          [k] ip_rcv_finish_core.isra.19
 1.28%  ksoftirqd/1  [kernel.kallsyms]          [k] ip_rcv_core.isra.20
 1.18%  ksoftirqd/1  [kernel.kallsyms]          [k] page_frag_free
 1.16%  ksoftirqd/1  [kernel.kallsyms]          [k] ip_finish_output2
```

Forwarding Results:NFOFF

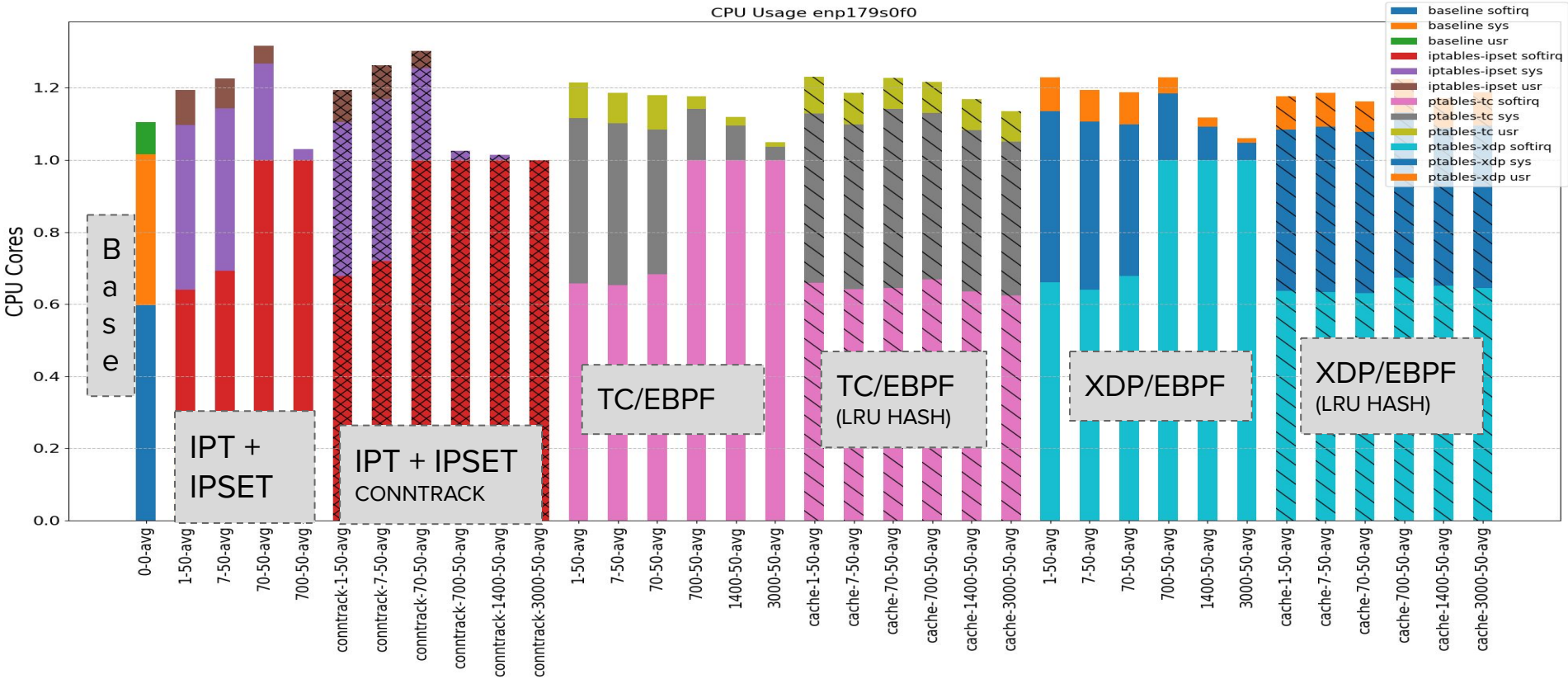
- Ruleset Linear iteration as expected got worse with rules
  - TC/XDP ebpf did better than IPtables/IPSet
  - IPtables/IPSet results after 700 ruleset intolerable
  - No difference between XDP and TC because packets go to stack
- Caching improved the performance of linear search
  - After 70 rulesets IPtables conntracking was >=3x than without
  - Conntracking still deteriorated with increasing rulesets
    - Likely the effect of the expensive linear iptables rules scan
  - Ebpf both at TC and XDP with caching outperformed conntracking
    - After 700 rulesets by 2x

Host Ingress NF compiled Into Kernel

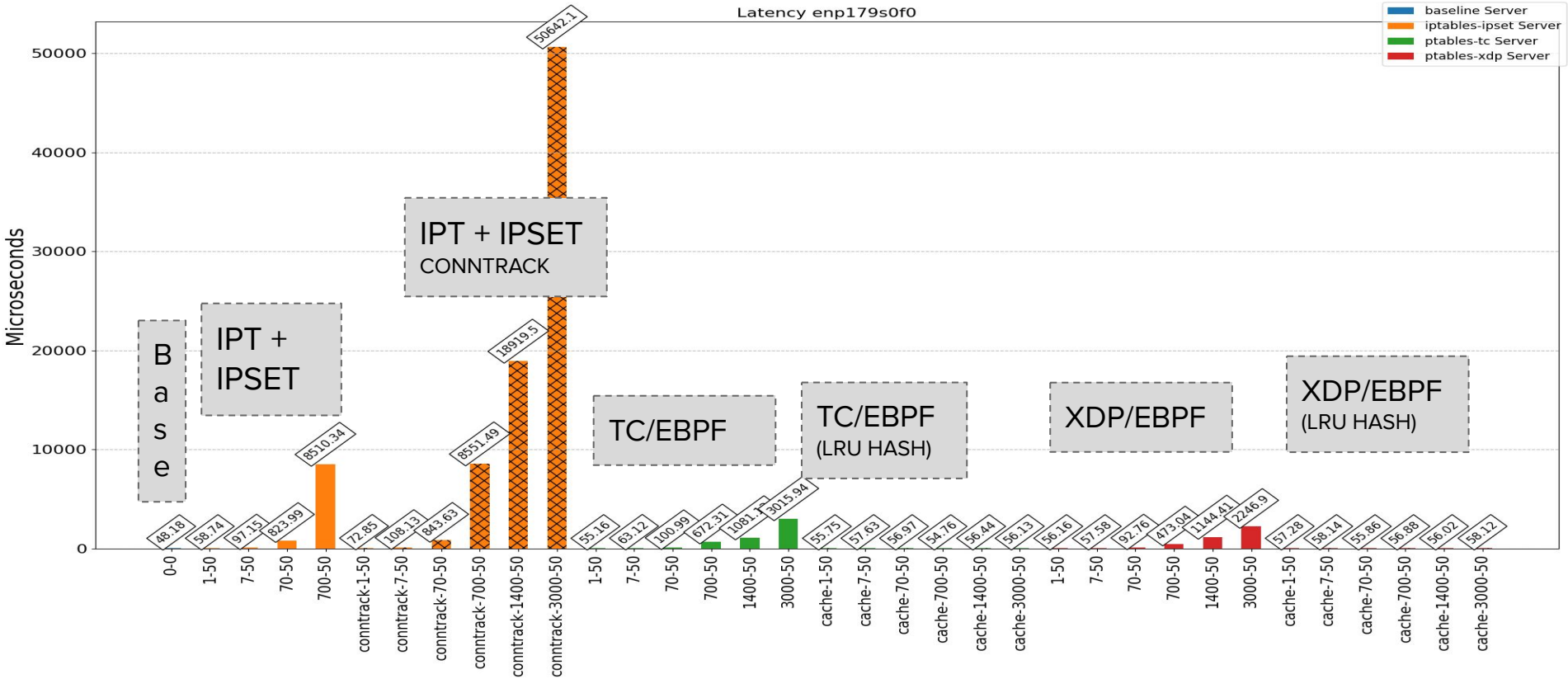
Host Ingress NFON: PPS (Higher Better)



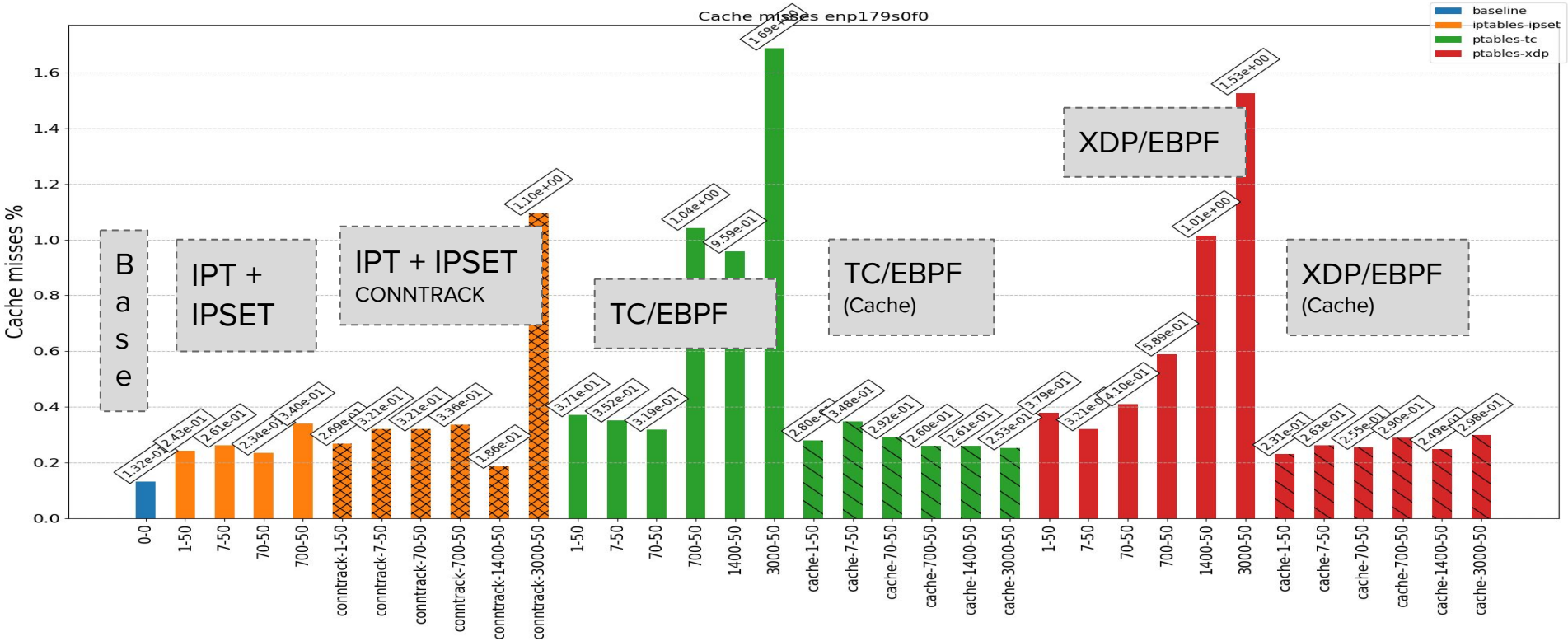
Host Ingress NFON: CPU (Lower Better)



Host Ingress NFON: Latency (Lower Better)

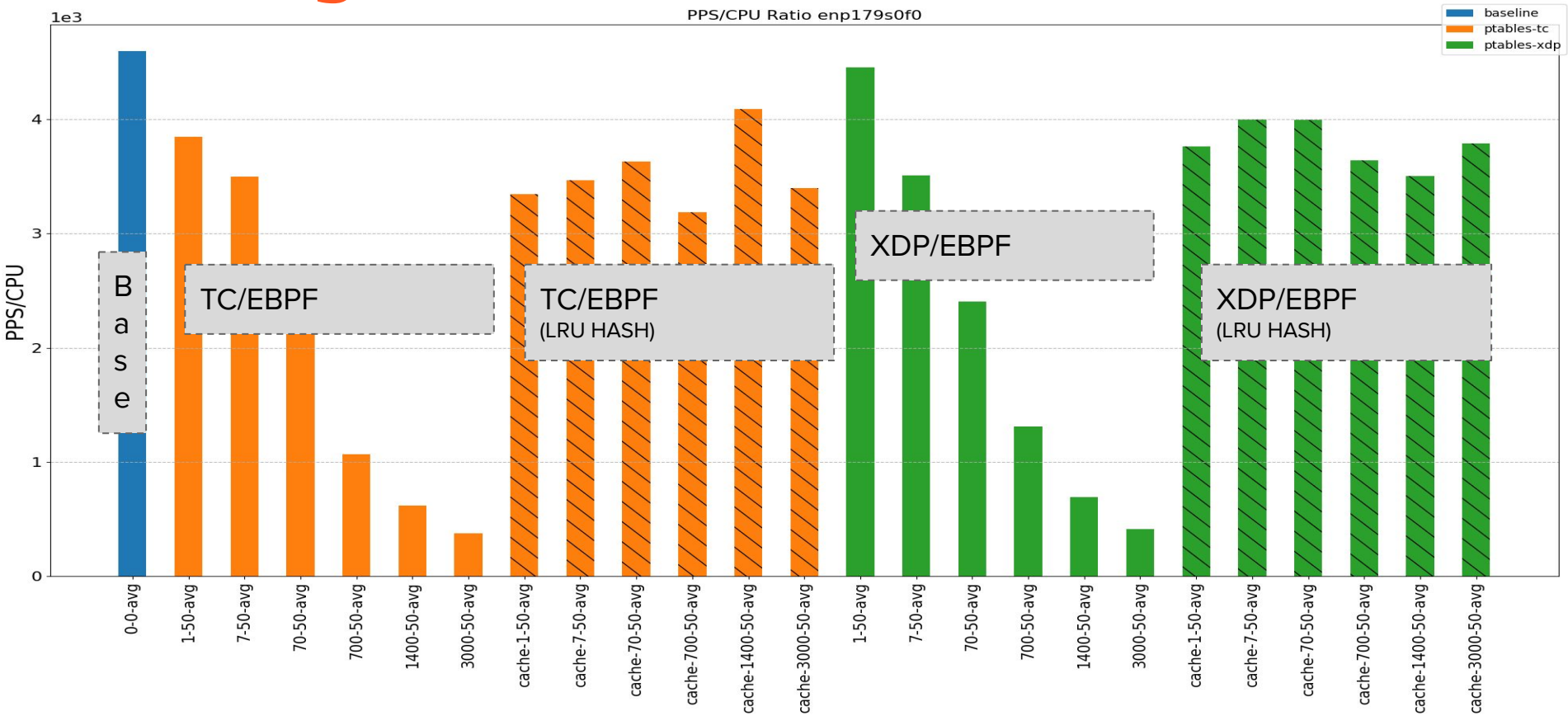


Host Ingress NFON: CPU Cache Misses (Lower Better)

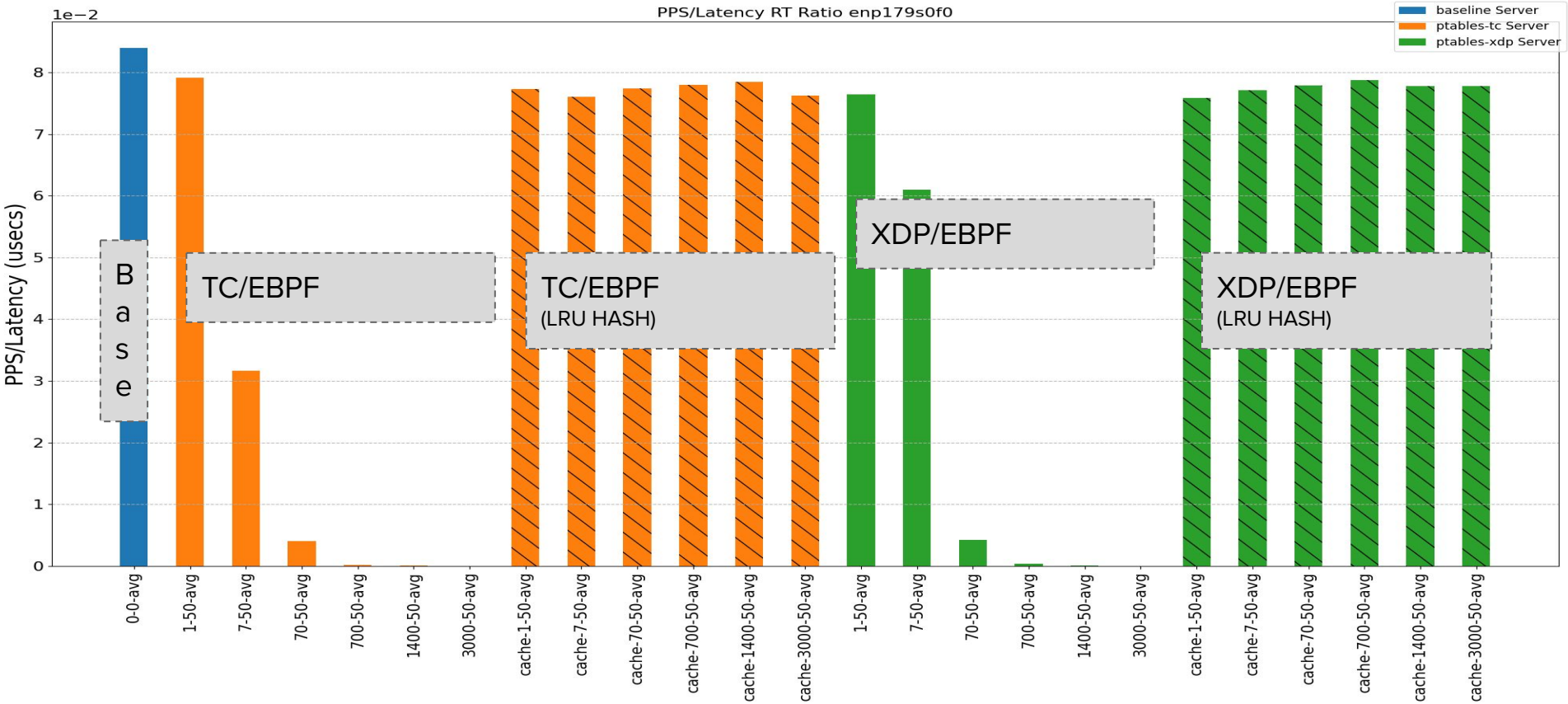


Host Ingress NF compiled out of Kernel

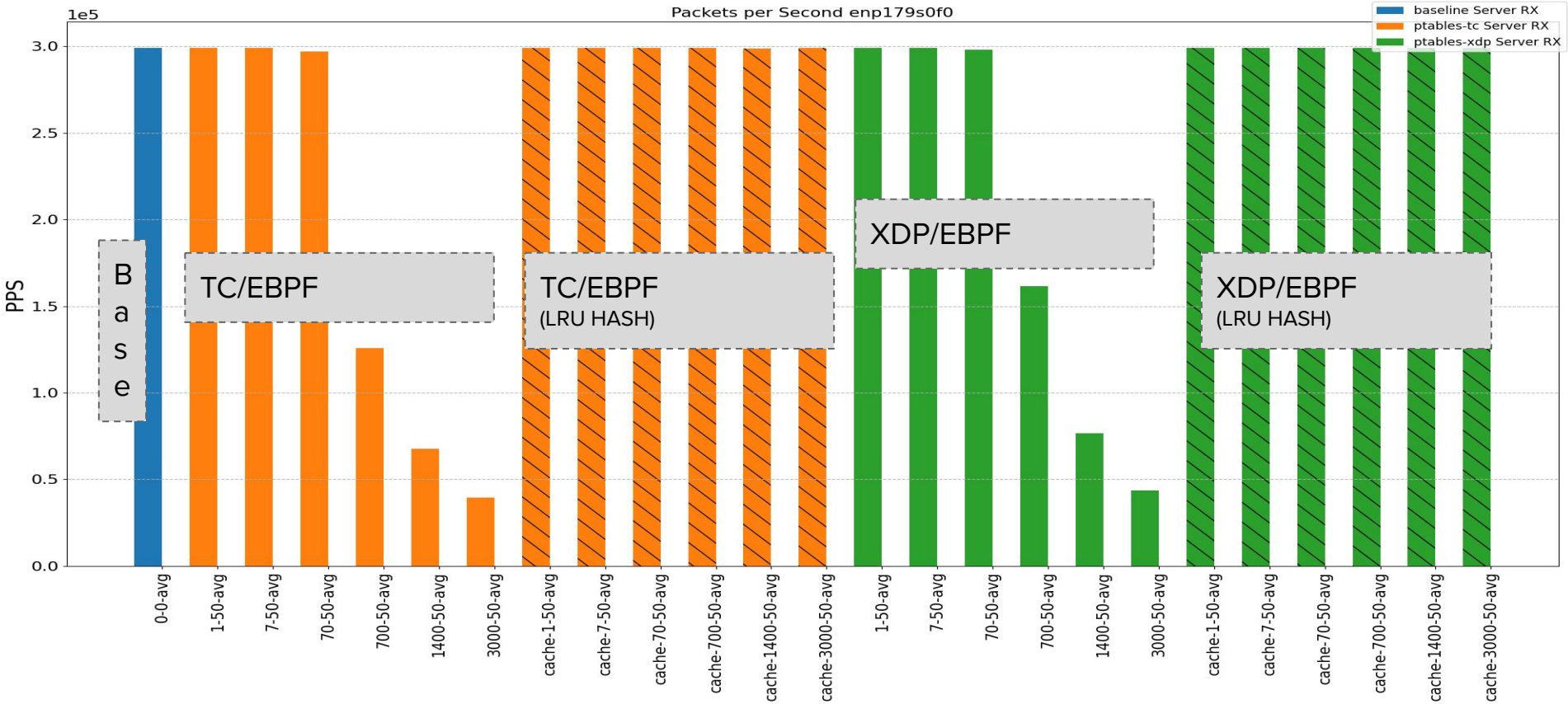
Host Ingress NFOFF: PPS/CPU (Higher Better)



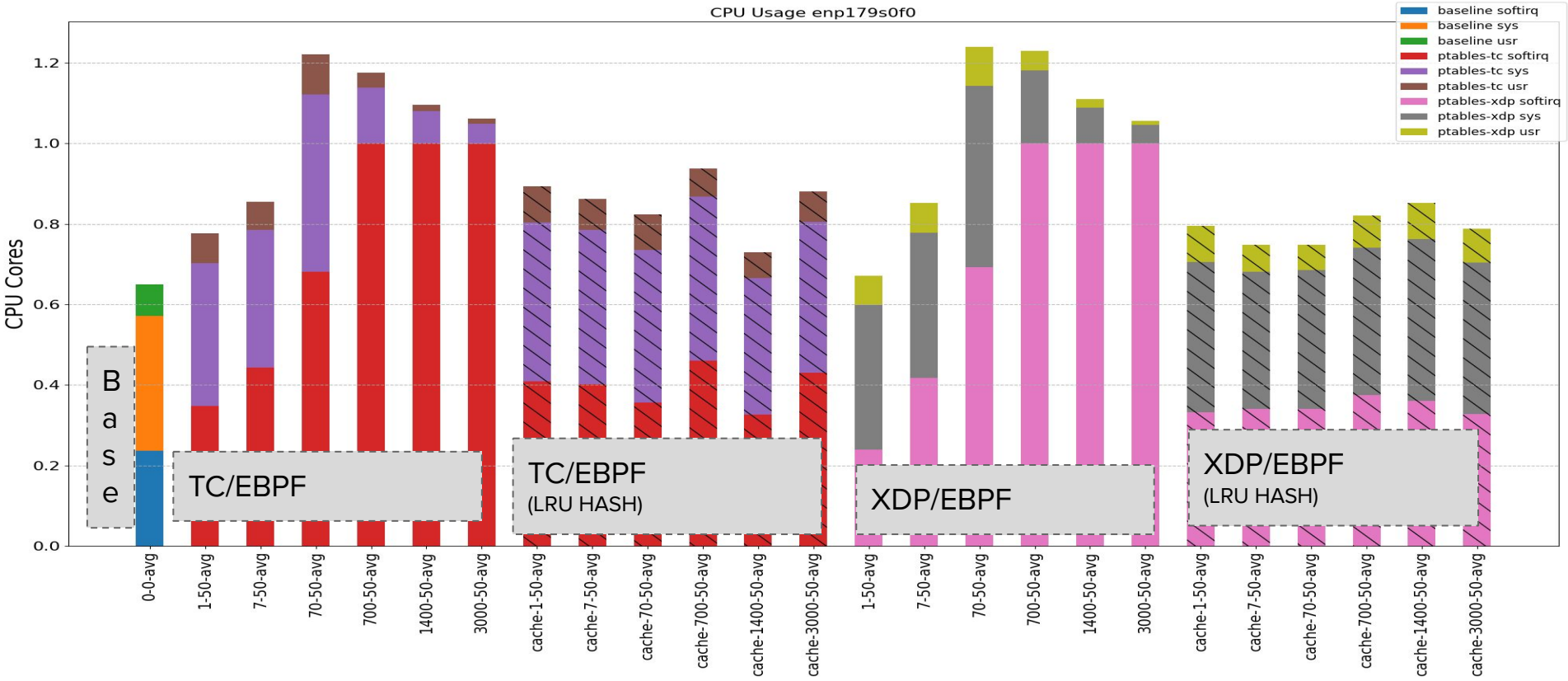
Host Ingress NFOFF: PPS/Latency (Higher Better)



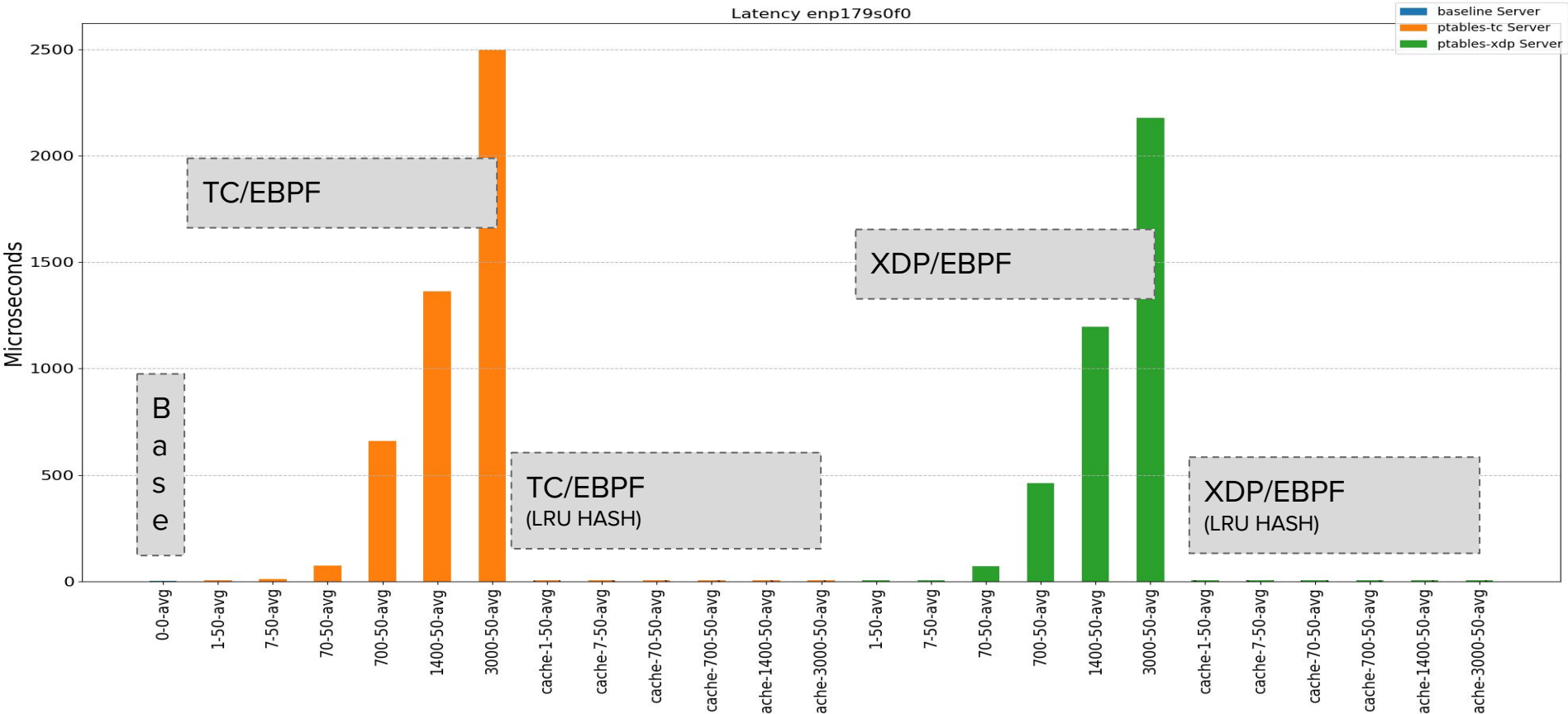
Host Ingress NFOFF: PPS (Higher Better)



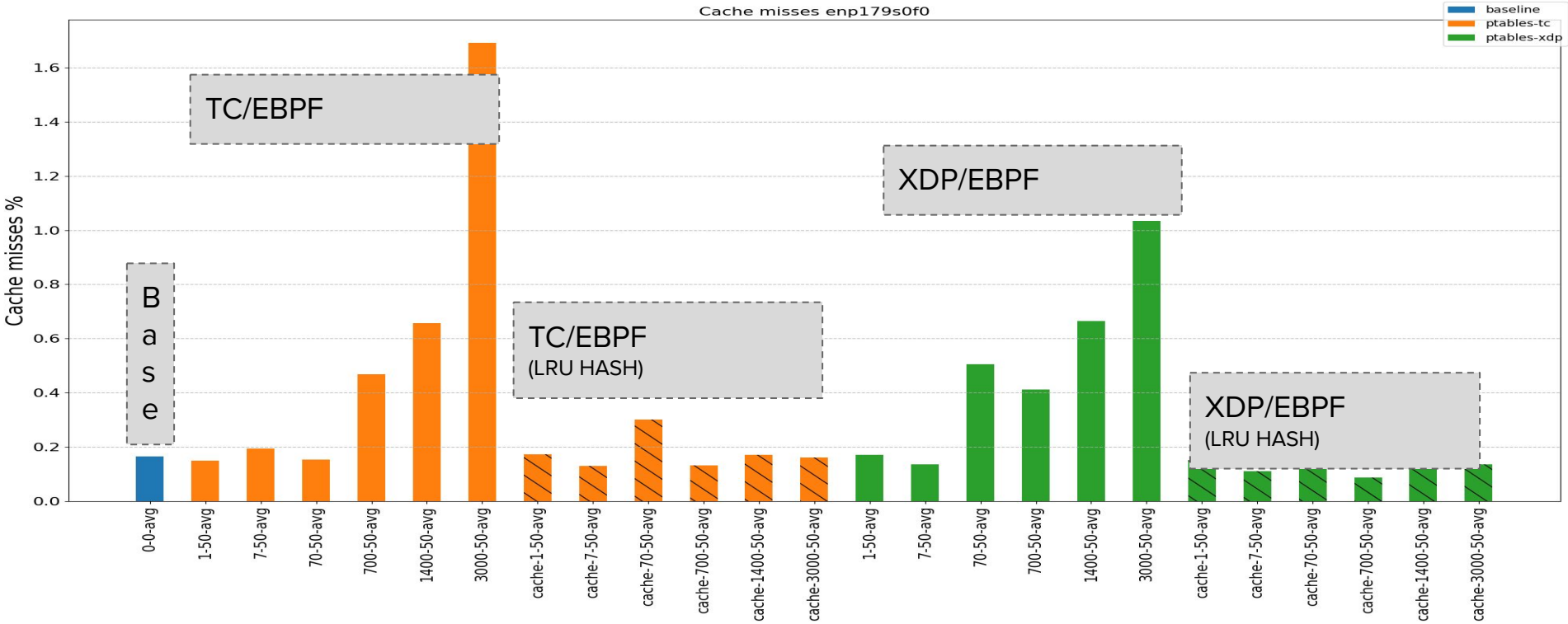
Host Ingress NFOFF: CPU(Lower Better)



Host Ingress NFOFF: Latency(Lower Better)

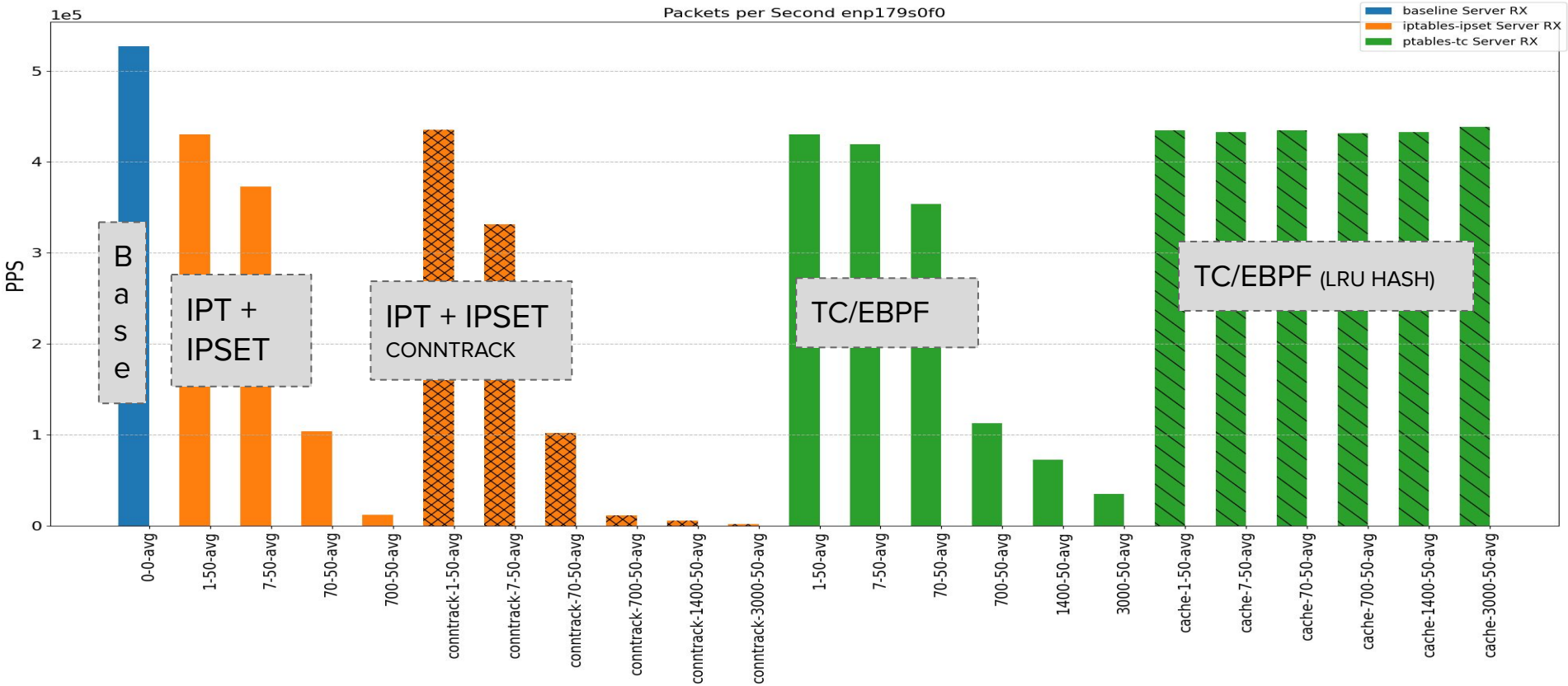


Host Ingress NFOFF: CPU Cache Misses(Lower Better)

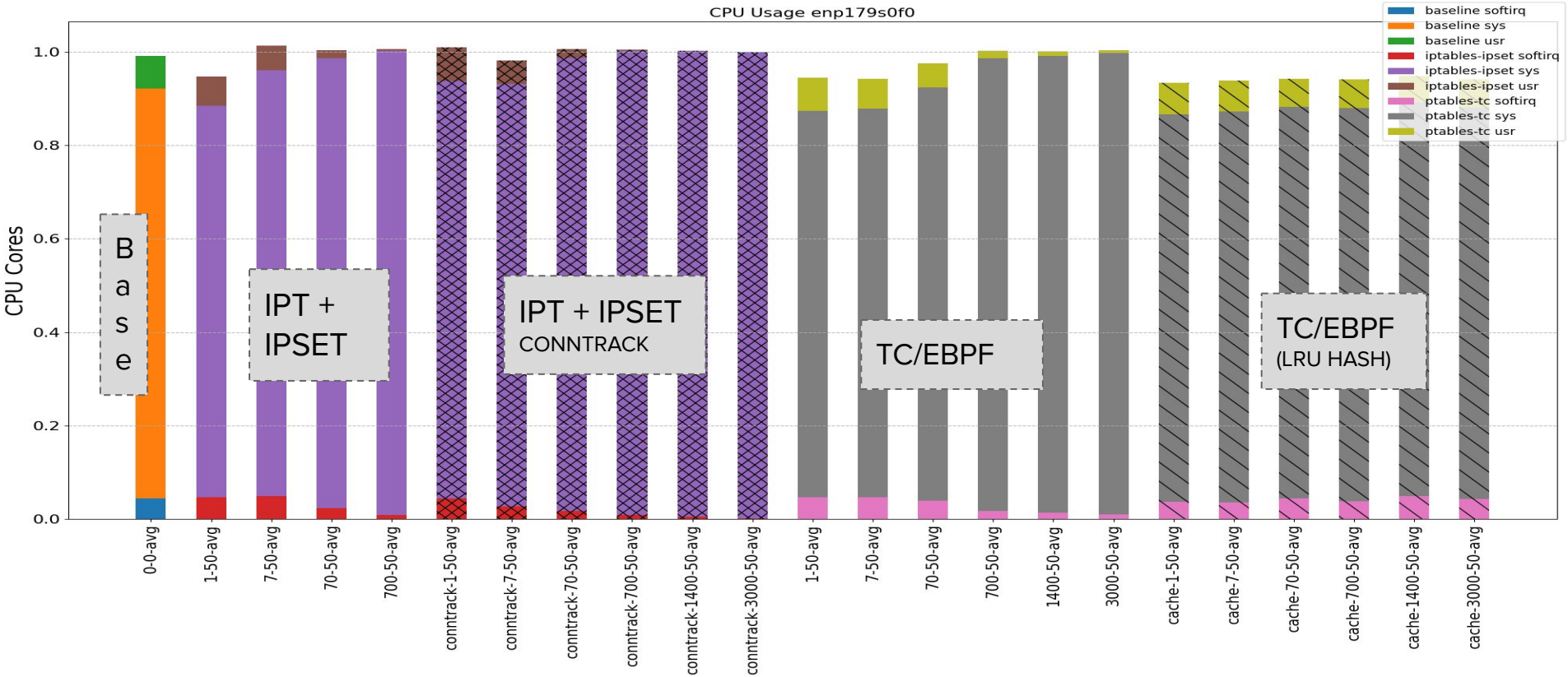


Host Egress NF compiled Into Kernel

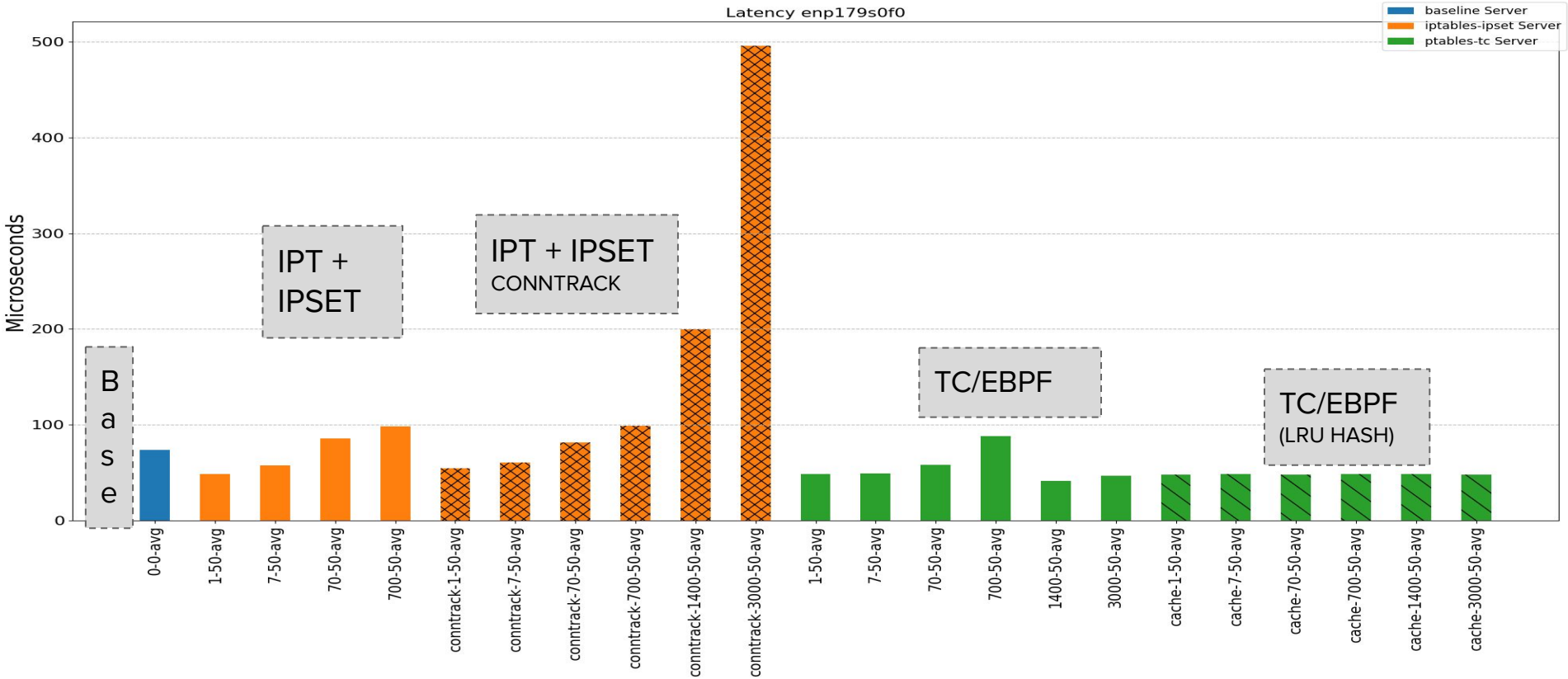
Host Egress NFON: PPS (Higher Better)



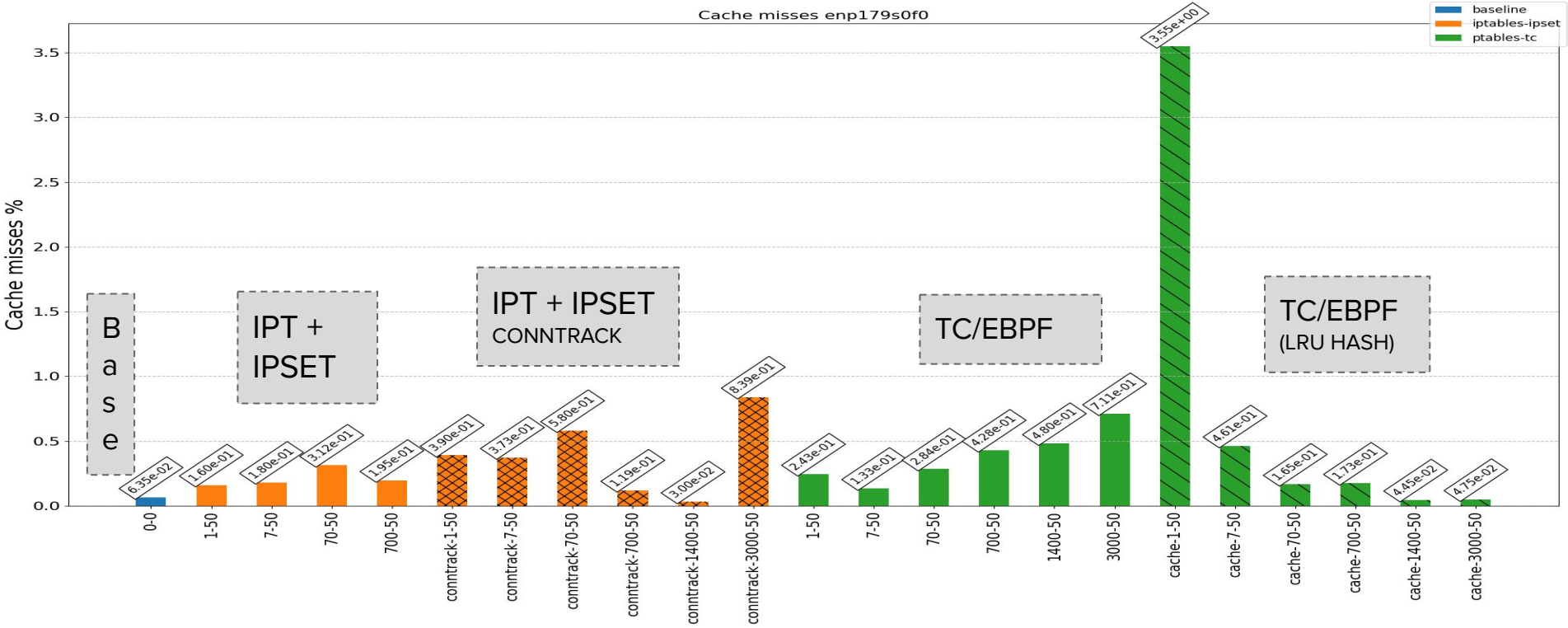
Host Egress NFON: CPU (Lower Better)



Host Egress NFON: Latency (Lower Better)

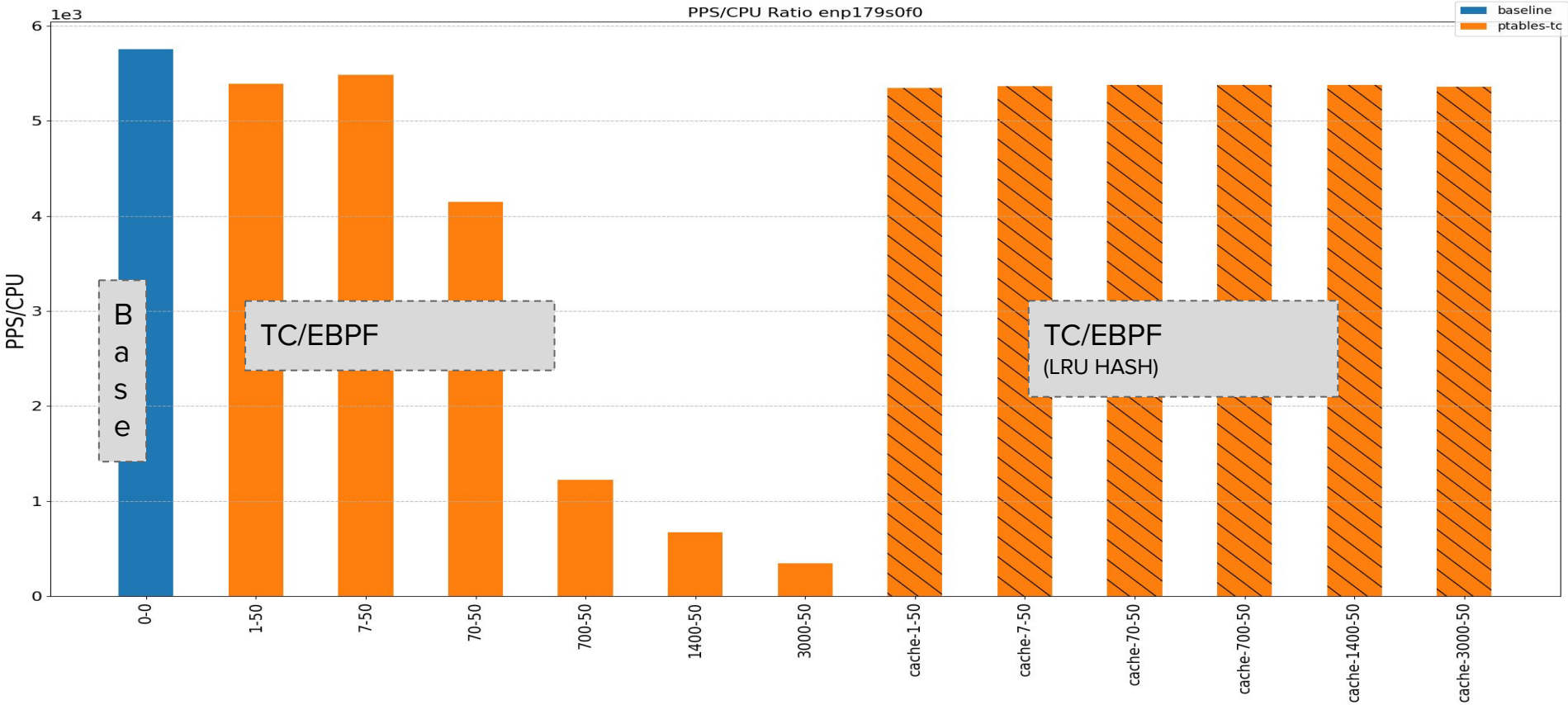


Host Egress NFON: CPU Cache Misses (Lower Better)

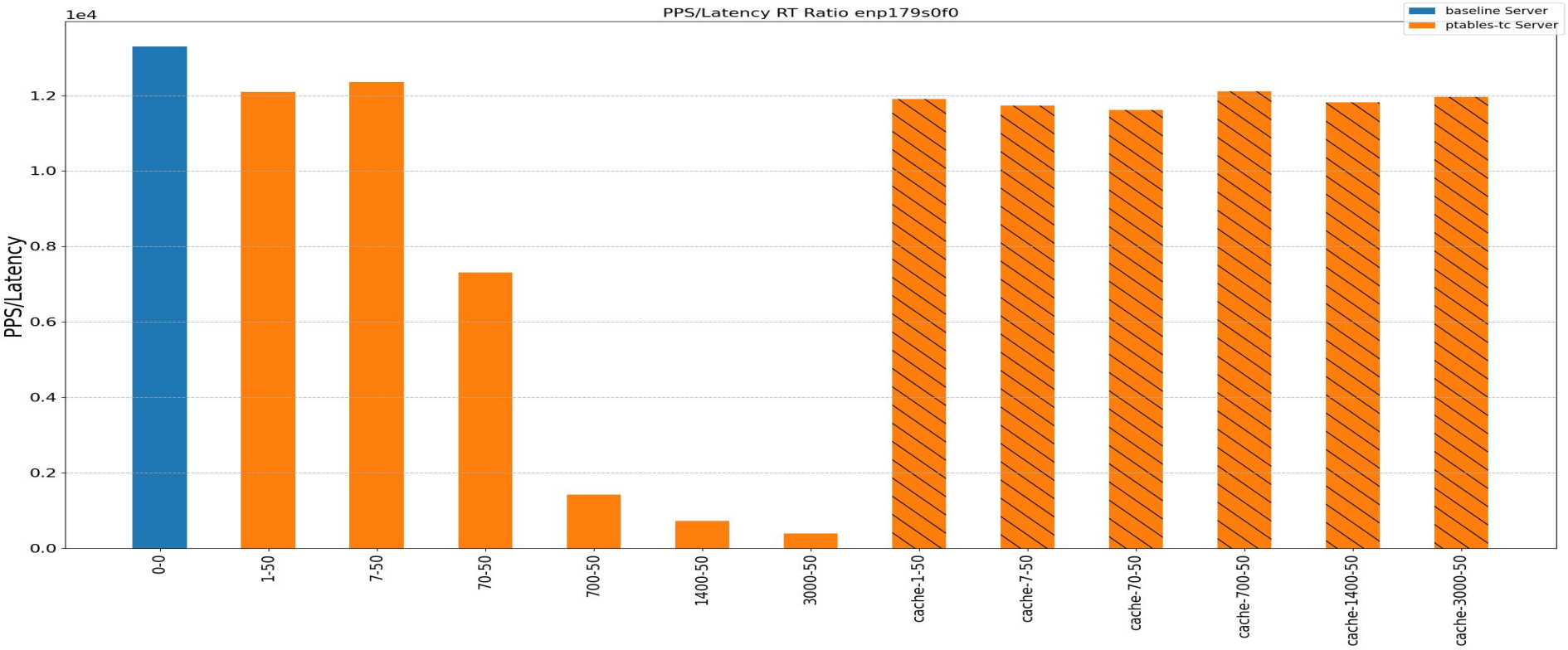


Host Egress NF compiled out of Kernel

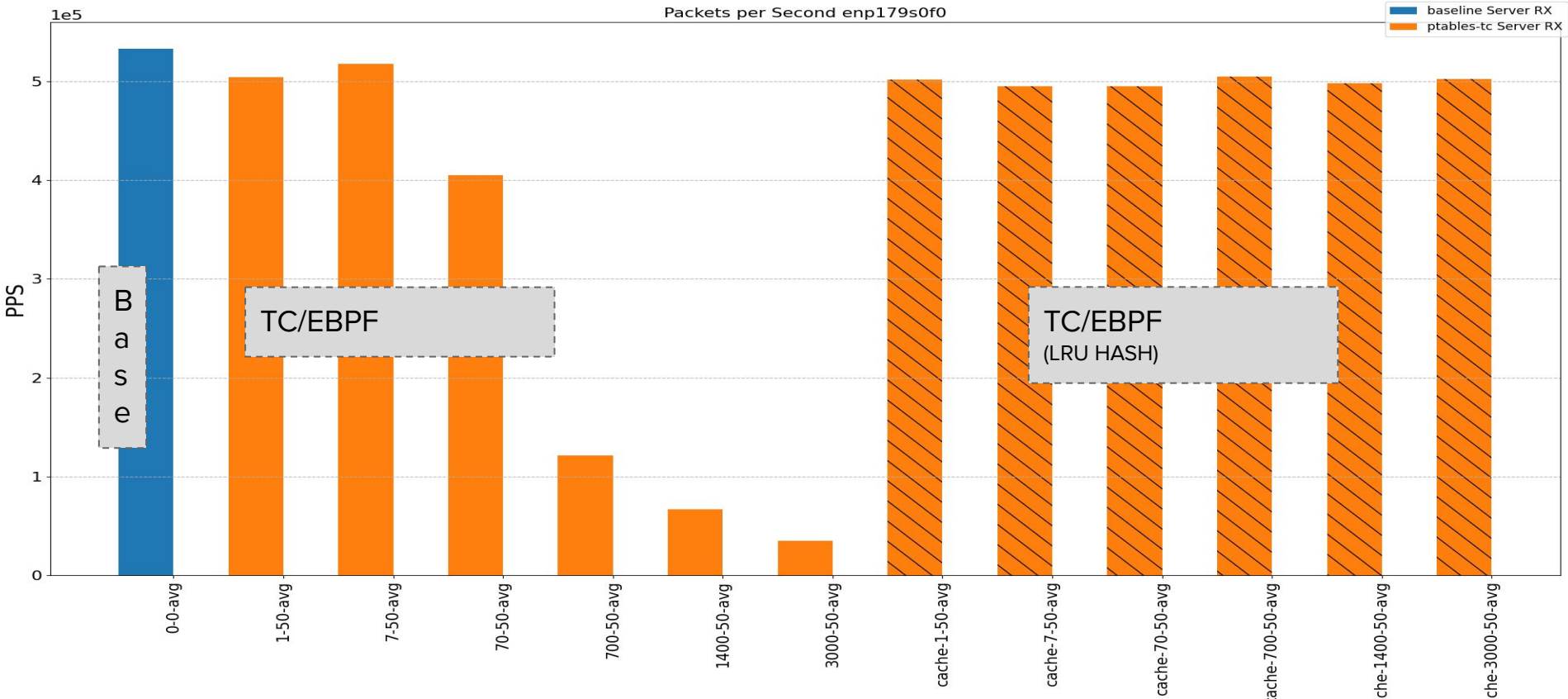
Host Egress NFOFF: PPS/CPU (Higher Better)



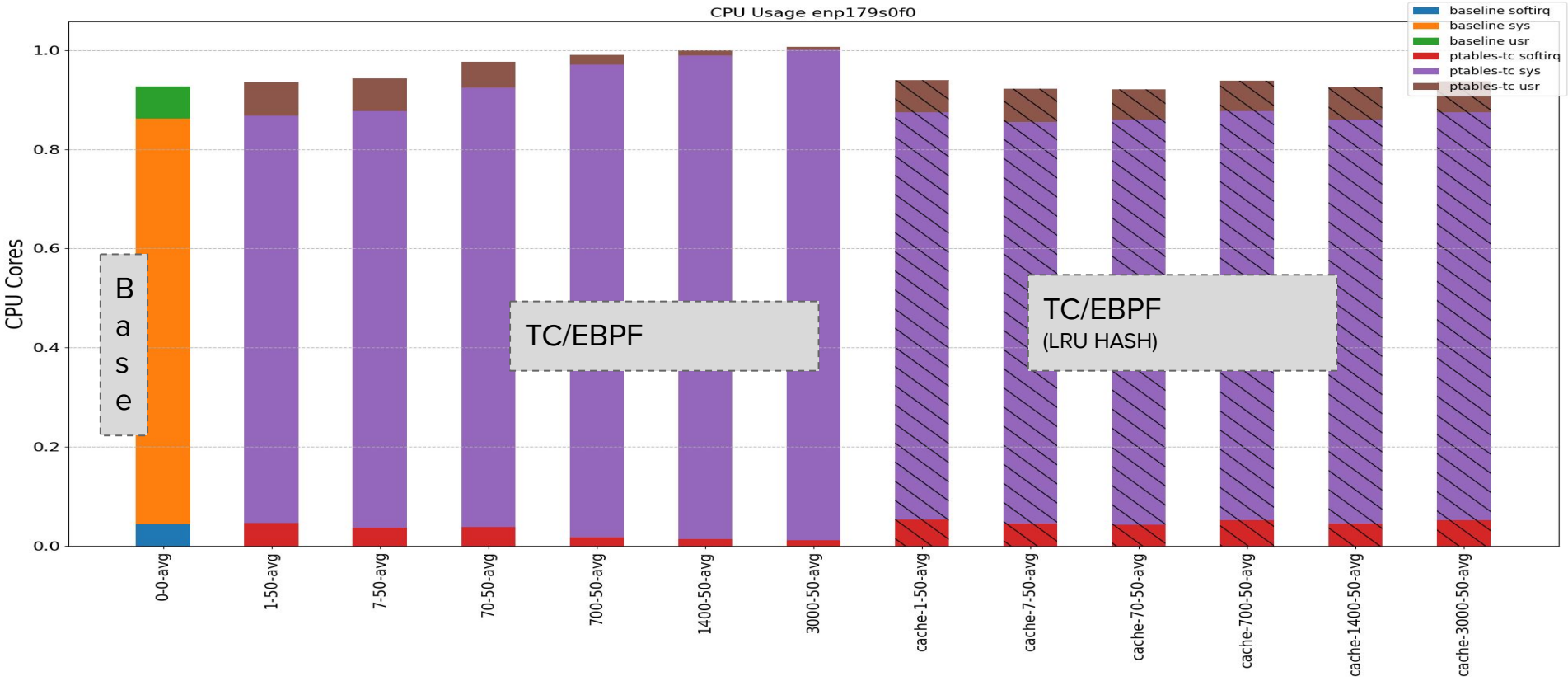
Host Egress NFOFF: PPS/Latency (Higher Better)



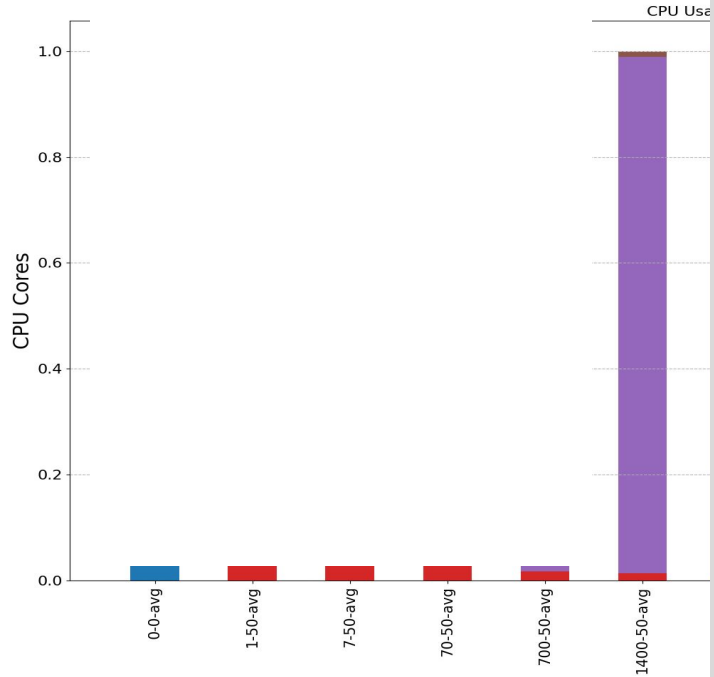
Host Egress NFOFF: PPS (Higher Better)



Host Egress NFOFF: CPU (Lower Better)



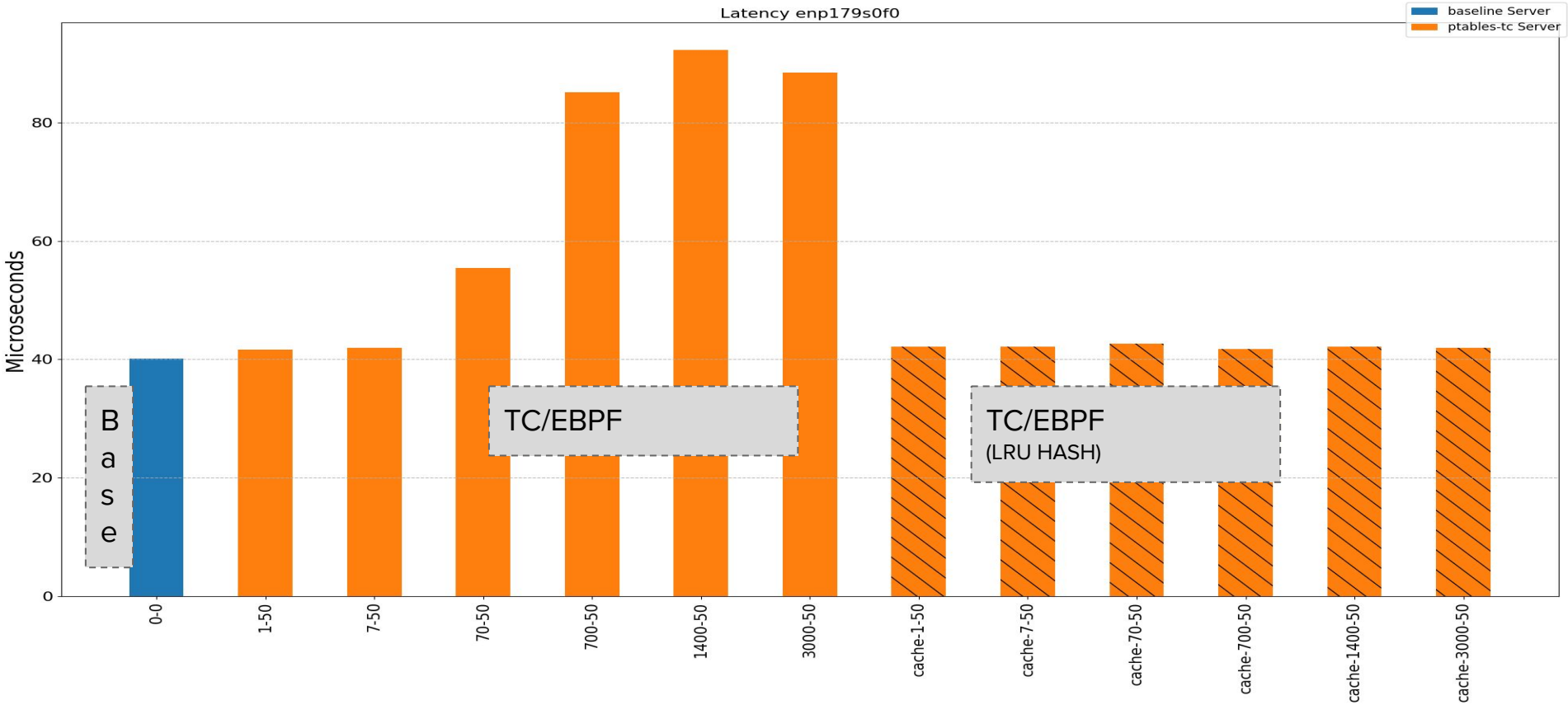
Host Egress NFOFF: CPU Mis-accounting?




```
86.73%  udpsend.tsk  bpf_prog_9ec5c00f80f63e94  [k] bpf_prog_9ec5c00f80f63e94
 1.36%  udpsend.tsk  [kernel.kallsyms]          [k] _raw_spin_lock
 0.76%  udpsend.tsk  [kernel.kallsyms]          [k] fib_table_lookup
 0.65%  udpsend.tsk  [kernel.kallsyms]          [k] copy_user_enhanced_fast_string
 0.57%  udpsend.tsk  [i40e]                  [k] i40e_lan_xmit_frame
 0.40%  udpsend.tsk  [kernel.kallsyms]          [k] ip_idents_reserve
```

Note: CPU cost on kernel code accounted to app?

Host Egress NFOFF: Latency (Lower Better)



Host Egress NFOFF: CPU Cache Misses (Lower Better)

